Anatomía de un LLM

Fundamentos matemáticos de los modelos de lenguaje

Fase III · Secuencias y Transformer

Módulo 15

Construye tu GPT

Entrenamiento, evaluación y generación de un Transformer decoder

---

Apuntes del curso

Edición de octubre de 2026

Presentación del módulo

Los módulos 12 a 14 han presentado por separado las piezas de un gran modelo de lenguaje: el tokenizador, la atención y el bloque Transformer. Este módulo las reúne en un modelo completo, un *mini-GPT* de unos 10^5 parámetros, y estudia todo lo que hace falta para entrenarlo y usarlo: cómo se convierte un corpus en ejemplos de entrenamiento, qué función se minimiza, con qué optimizador y con qué calendario de la tasa de aprendizaje, cómo se mide el progreso y se guardan puntos de control, cómo se leen las curvas de pérdida y cómo se genera texto con temperatura, top-k y top-p. La receta es, a escala reducida, la que documentan los modelos abiertos como GPT-2 y LLaMA; los detalles de entrenamiento de Claude, Gemini y Grok no se han publicado.

Objetivos

Al finalizar el módulo, el estudiante debe ser capaz de:

1. construir las ventanas de contexto y los lotes a partir de un corpus tokenizado, contar las predicciones que contienen y definir la pérdida de entropía cruzada y la perplejidad;
2. derivar el gradiente de la entropía cruzada respecto de los logits, demostrar la invariancia de la softmax frente a traslaciones y justificar el valor $\ln K$ de la pérdida inicial;
3. describir la arquitectura completa de un Transformer decoder, contar sus parámetros y demostrar que la máscara causal hace que una sola pasada produzca las T predicciones de una ventana;
4. definir AdamW, el calendario de calentamiento lineal y decaimiento coseno y el recorte del gradiente por norma global, y demostrar sus propiedades principales;
5. estimar la pérdida de validación, diseñar puntos de control, interpretar una curva de pérdida con sus tres referencias y demostrar que la entropía del proceso generador es una cota inferior de la pérdida esperada;
6. definir la temperatura, el muestreo top-k y el muestreo de núcleo (top-p), demostrar que la entropía de la distribución crece con la temperatura y razonar el efecto de los hiperparámetros principales.

Requisitos previos

Del módulo 1 se usan la regla de la cadena, la softmax con temperatura, la entropía cruzada y la perplejidad; del módulo 6, la divergencia de Kullback–Leibler y la desigualdad de Gibbs; del módulo 7, Adam, AdamW y el calendario coseno; del módulo 8, la softmax con entropía cruzada; del módulo 10, la diferenciación automática y PyTorch; del módulo 11, la normalización de capa, las conexiones residuales y el recorte del gradiente; del módulo 12, la tokenización; del módulo 13, la atención con máscara causal y multicabeza, y del módulo 14, el bloque Transformer y la regla $C \approx 6ND$.

Organización

Tema	Contenido	Fuentes principales
1	Datos, ventanas y objetivo de entrenamiento	[11], [16], [15]
2	La arquitectura mini-GPT	[10], [13], [11], [16], [12]
3	Optimización: AdamW, calendario y recorte	[11], [9], [8], [12], [4]
4	Evaluación, puntos de control y curvas de pérdida	[4], [11], [1], [7]
5	Generación e hiperparámetros	[11], [2], [5], [4]

Cada tema termina con una lista de lecturas y con ejercicios graduados en tres niveles: $\star$ comprensión, $\star\star$ cálculo y $\star\star\star$ demostración. Los ejercicios marcados con *Fuente* están adaptados de cursos y libros de texto universitarios; el enunciado se ha redactado de nuevo y, en algunos casos, se han cambiado los datos para que puedan resolverse a mano. Las soluciones completas, paso a paso, están en el apéndice A.

Sobre las fuentes

El texto es de redacción propia. La organización sigue la tarea 1 (*basics*) del curso CS336 de Stanford, «Language Modeling from Scratch» [11], en la que se programa un modelo de lenguaje Transformer completo: el §4 (entropía cruzada, AdamW, calendario de la tasa y recorte del gradiente), el §5 (cargador de datos, puntos de control y bucle de entrenamiento) y el §6 (decodificación con temperatura y top-p). Se complementa con las clases 5 («Transformers») y 7 («Pretraining») del curso CS224N [15], con los capítulos 9 y 11 de Zhang *et al.* [16] (§9.3 sobre la partición de secuencias, §9.5 sobre recorte y decodificación, §11.9.3 sobre los modelos decoder) y con los capítulos 7, 8 y 11 de Goodfellow, Bengio y Courville [4] para la parada temprana, los algoritmos de optimización y la metodología práctica. Los métodos se toman de sus artículos originales: Vaswani *et al.* [13] para el Transformer, Radford *et al.* [10] para GPT-2, Touvron *et al.* [12] para la receta de entrenamiento de LLaMA, Loshchilov y Hutter para AdamW [9] y para el calendario coseno [8], Fan, Lewis y Dauphin [2] para el muestreo top-k y Holtzman *et al.* [5] para el muestreo de núcleo. El código de referencia de la práctica sigue la estructura del repositorio nanoGPT de Karpathy [6]. Los resultados numéricos de entrenamiento proceden del cuaderno de prácticas del módulo, ejecutado con la semilla indicada en él. Sobre los modelos comerciales (Claude, Gemini, Grok) solo se usa información pública.

Notación

Símbolo	Significado
$\mathcal{V}$, $K = \lvert\mathcal{V}\rvert$	vocabulario y su tamaño
$u_{1:n}$	corpus tokenizado de n tokens
T, B	longitud de la ventana de contexto y tamaño del lote
$x_{1:T}$, $y_{1:T}$	ventana de entrada y ventana objetivo, con $y_t = x_{t+1}$
d, L, n_h, $d_k = d/n_h$	dimensión del modelo, número de bloques, de cabezas y dimensión por cabeza
$\boldsymbol{Q}, \boldsymbol{K}, \boldsymbol{V}$, $\boldsymbol{M}$	matrices de consultas, claves y valores, y máscara causal
$z \in \mathbb{R}^K$, $p = \mathrm{softmax}(z)$	logits y distribución predicha en una posición
ℓ, $\mathcal{L}$	pérdida de una posición y pérdida media
θ, N	parámetros del modelo y su número
s, S, W	paso de entrenamiento, número total de pasos y pasos de calentamiento
η_s, $\eta_{\max}$, $\eta_{\min}$	tasa de aprendizaje en el paso s y sus valores máximo y mínimo
λ, β_1, β_2	decaimiento de pesos y coeficientes de los momentos de AdamW
c, clip_c	umbral y operador de recorte del gradiente
τ, k, ρ	temperatura, parámetro de top-k y umbral de top-p
$H(p)$, $H(p,q)$, $D_{\mathrm{KL}}(p\Vert q)$	entropía, entropía cruzada y divergencia de Kullback–Leibler (en nats)

Índice

Presentación del módulo **ii**

1 Datos, ventanas y objetivo de entrenamiento **1**

1.1 El corpus como secuencia de tokens . . . 1
1.2 Ventanas de contexto y lotes . . . 2
1.3 La pérdida de entropía cruzada . . . 2
1.4 Gradiente y estabilidad numérica . . . 3
Ejercicios del tema 1 . . . 4

2 La arquitectura mini-GPT **6**

2.1 Definición del modelo . . . 6
2.2 La máscara causal . . . 7
2.3 Recuento de parámetros . . . 9
2.4 Del mini-GPT a los modelos abiertos . . . 9
Ejercicios del tema 2 . . . 10

3 Optimización: AdamW, calendario y recorte **12**

3.1 El paso de entrenamiento . . . 12
3.2 AdamW . . . 13
3.3 Calentamiento lineal y decaimiento coseno . . . 13
3.4 Recorte del gradiente por norma global . . . 14
3.5 El coste del entrenamiento . . . 15
Ejercicios del tema 3 . . . 16

4 Evaluación, puntos de control y curvas de pérdida **17**

4.1 Estimar la pérdida . . . 17
4.2 Puntos de control . . . 18
4.3 La cota inferior de la pérdida . . . 18
4.4 Lectura de las curvas de pérdida . . . 19
Ejercicios del tema 4 . . . 20

5 Generación e hiperparámetros **22**
5.1 Generación autorregresiva . . . 22
5.2 Temperatura . . . 22
5.3 Muestreo top-k y top-p . . . 23
5.4 Efecto de los hiperparámetros . . . 24
Ejercicios del tema 5 . . . 26

A Soluciones de los ejercicios **27**
A.1 Tema 1 . . . 27
A.2 Tema 2 . . . 30
A.3 Tema 3 . . . 33
A.4 Tema 4 . . . 36
A.5 Tema 5 . . . 38

Bibliografía **42**

Tema 1

Datos, ventanas y objetivo de entrenamiento

Un modelo de lenguaje autorregresivo se entrena con una única secuencia larga de enteros, de la que se extraen ventanas de longitud fija. Este tema formaliza esa preparación de los datos, define la función que se minimiza (la entropía cruzada media sobre todas las posiciones de un lote) y demuestra tres propiedades de esa función que se usan durante todo el módulo: su gradiente respecto de los logits, su invariancia frente a traslaciones de los logits, que permite calcularla de forma estable, y su valor cuando la predicción es uniforme. Se sigue el §4.1 y el §5.1 de la tarea 1 de CS336 [11], el §9.3.3 de [16] y la clase 7 de CS224N [15].

1.1 El corpus como secuencia de tokens

Definición 1.1 (Corpus tokenizado). Sea $\mathcal{V}$ un vocabulario de tamaño K, identificado con $\{0, 1, \ldots, K-1\}$. Un *corpus tokenizado* es una secuencia $u_{1:n} = (u_1, \ldots, u_n) \in \mathcal{V}^n$. Si el texto original está formado por varios documentos, se concatenan en una sola secuencia, separados por un token delimitador.

La concatenación es una decisión de ingeniería con una justificación precisa: todas las ventanas tienen la misma longitud y no hace falta rellenar con símbolos de relleno (*padding*), que consumen cómputo sin aportar señal [11, §5.1]. El precio es que una ventana puede cruzar la frontera entre dos documentos; el delimitador permite al modelo aprender que lo que sigue no depende de lo anterior.

Ejemplo 1.2 (El corpus de la práctica). El cuaderno del módulo genera 200 líneas de texto sintético en español con cuatro plantillas: partes del tiempo, recetas, diálogos y descripciones de objetos inventados. Por ejemplo:

```
objeto. una taza blanca de barro. sirve para llevar libros.
receta. asa calabaza con cebolla durante diez minutos. añade sal y sirve templado.
```

El corpus tiene $n = 16\,066$ caracteres y 46 caracteres distintos. Se tokeniza por caracteres: $\mathcal{V}$ es la lista ordenada de caracteres distintos, de modo que $K = 46$, el salto de línea `\n` es el token 0 y el espacio el token 1. Un tokenizador BPE (módulo 12) acortaría las secuencias, pero con un corpus tan pequeño apenas hay pares frecuentes que fusionar.

Un corpus sintético tiene una ventaja metodológica: se conoce el proceso que lo genera. Se sabe, por ejemplo, que tras «receta.» viene uno de seis verbos y que «un reloj» exige «negro» y no «negra». Esto permite juzgar qué ha aprendido el modelo y calcular la pérdida mínima alcanzable (tema 4).

1.2 Ventanas de contexto y lotes

Definición 1.3 (Ventana de contexto). Sea $T \geq 1$ la *longitud de contexto*. Para cada $i \in \{1, \ldots, n-T\}$, la *ventana* que empieza en i es el par

$$x = (u_i, u_{i+1}, \ldots, u_{i+T-1}), \qquad y = (u_{i+1}, u_{i+2}, \ldots, u_{i+T}),$$

es decir, $y_t = x_{t+1}$ para $t < T$ e $y_T = u_{i+T}$. La secuencia y es la *ventana objetivo*.

Proposición 1.4. *Un corpus de n tokens tiene exactamente $n - T$ ventanas de longitud T, y cada una contiene T problemas de predicción: predecir y_t a partir del prefijo $x_{1:t}$, para $t = 1, \ldots, T$.*

Demostración. La ventana que empieza en i necesita los tokens $u_i, \ldots, u_{i+T}$, es decir, $T + 1$ tokens consecutivos. Esto exige $i \geq 1$ e $i + T \leq n$, luego $i \in \{1, \ldots, n - T\}$, que tiene $n - T$ elementos. En la posición t, el contexto disponible es $x_{1:t} = u_{i:i+t-1}$ y el token que lo sigue en el corpus es $u_{i+t} = y_t$. ∎

Ejemplo 1.5. Con el texto «sal y sol\n» ($n = 10$) y $T = 4$ hay $10 - 4 = 6$ ventanas:

Inicio	x	y	Inicio	x	y
1	`sal␣`	`al␣y`	4	`␣y␣s`	`y␣so`
2	`al␣y`	`l␣y␣`	5	`y␣so`	`␣sol`
3	`l␣y␣`	`␣y␣s`	6	`␣sol`	`sol\n`

En la primera ventana el modelo debe predecir «a» tras «s», «l» tras «sa», el espacio tras «sal» e «y» tras «sal␣». En total hay $6 \cdot 4 = 24$ predicciones en un texto de 10 caracteres.

Definición 1.6 (Lote). Un *lote* de tamaño B es un par de matrices de enteros $X, Y \in \mathcal{V}^{B \times T}$ cuyas filas son B ventanas, con inicios elegidos de forma independiente y uniforme en $\{1, \ldots, n - T\}$. Un lote contiene $B \cdot T$ predicciones.

Con $B = T = 32$, cada paso de entrenamiento ve 1024 predicciones; en 800 pasos son 819 200, unas 57 veces los 14 459 tokens de entrenamiento del ejemplo 1.2. Con tan pocos datos es inevitable repetir; los grandes modelos, en cambio, ven cada token una vez o pocas veces (módulo 16).

Observación 1.7 (Partición en entrenamiento y validación). El conjunto de validación se obtiene cortando el corpus *por posición*: el 90 % inicial (14 459 tokens) se usa para entrenar y el 10 % final (1 607 tokens) para validar. Si se eligieran ventanas al azar para cada conjunto, una ventana de validación podría solaparse con una de entrenamiento y la pérdida de validación subestimaría el error de generalización.

La alternativa al muestreo independiente de inicios es partir el corpus en trozos consecutivos sin solape, con un desplazamiento inicial aleatorio en cada época, como hace [16, §9.3.3]. El ejercicio 1.3 compara ambas estrategias.

1.3 La pérdida de entropía cruzada

En cada posición t de cada ventana, el modelo devuelve un vector de *logits* $z_t \in \mathbb{R}^K$ y la distribución predicha $p_t = \mathrm{softmax}(z_t)$, con $p_{t,j} = e^{z_{t,j}} / \sum_a e^{z_{t,a}}$.

Definición 1.8 (Pérdida de entrenamiento). La pérdida de una posición con objetivo $y \in \mathcal{V}$ y logits z es

$$\ell(z, y) = -\ln \operatorname{softmax}(z)_y.$$

La pérdida de un lote es la media sobre sus $B \cdot T$ posiciones,

$$\mathcal{L}(\theta) = \frac{1}{BT} \sum_{b=1}^{B} \sum_{t=1}^{T} \ell\big(z_{b,t}(\theta),\, Y_{b,t}\big),$$

donde $z_{b,t}(\theta)$ son los logits que el modelo de parámetros θ produce en la posición t de la fila b a partir de $X_{b,1:t}$.

La pérdida ℓ es la entropía cruzada entre la distribución degenerada en y y la distribución predicha [11, §4.1]. Su media sobre el corpus es la estimación de máxima verosimilitud del módulo 1 aplicada a un modelo con parámetros compartidos: minimizar $\mathcal{L}$ equivale a maximizar $\sum \ln P_\theta(y_t \mid x_{1:t})$.

Definición 1.9 (Perplejidad). Si una secuencia de m predicciones tiene pérdidas $\ell_1, \ldots, \ell_m$, su *perplejidad* es

$$\mathrm{PPL} = \exp\Big(\frac{1}{m} \sum_{i=1}^{m} \ell_i\Big).$$

La perplejidad es el número de opciones equiprobables entre las que el modelo duda en media: un modelo uniforme sobre K símbolos tiene perplejidad K. Es independiente de la base del logaritmo, porque $e^{\mathcal{L}} = 2^{\mathcal{L}/\ln 2}$. En la práctica del módulo, el modelo de bigramas del módulo 1 llega a 2,035 nats por carácter (PPL = 7,65) y el mini-GPT a 0,324 (PPL = 1,38).

1.4 Gradiente y estabilidad numérica

Proposición 1.10 (Gradiente de la entropía cruzada). *Sea* $p = \operatorname{softmax}(z)$. *Para todo* $i \in \mathcal{V}$,

$$\frac{\partial \ell(z, y)}{\partial z_i} = p_i - \mathbb{1}[i = y],$$

es decir, $\nabla_z \ell = p - e_y$, *donde* e_y *es el vector* one-hot *de* y.

Demostración. Por la definición de la softmax, $\ell(z, y) = -z_y + \ln \sum_a e^{z_a}$. La derivada del primer término respecto de z_i es $-\mathbb{1}[i = y]$. La del segundo es $e^{z_i} / \sum_a e^{z_a} = p_i$ por la regla de la cadena. ∎

Ejemplo 1.11. Sean $K = 3$, $z = (2, 1, 0)$ y el objetivo y el primero de los tres tokens: $\sum_a e^{z_a} = 7{,}389 + 2{,}718 + 1 = 11{,}107$, luego $p = (0{,}6652;\ 0{,}2447;\ 0{,}0900)$, $\ell = -\ln 0{,}6652 = 0{,}4076$ y $\nabla_z \ell = (-0{,}3348;\ 0{,}2447;\ 0{,}0900)$. El gradiente sube el logit correcto y baja los demás en proporción a su probabilidad; sus componentes suman cero.

El gradiente de la proposición 1.10 es la única señal que recibe todo el mini-GPT: la retropropagación (módulo 10) la transporta desde los logits hasta los embeddings.

Proposición 1.12 (Invariancia frente a traslaciones). *Para todo* $c \in \mathbb{R}$, $\operatorname{softmax}(z + c\mathbf{1}) = \operatorname{softmax}(z)$. *En consecuencia, con* $z^\star = \max_a z_a$,

$$\ell(z, y) = -(z_y - z^\star) + \ln \sum_a e^{\,z_a - z^\star},$$

y todos los exponentes del segundo miembro son menores o iguales que 0.

Demostración. $e^{z_j+c}/\sum_a e^{z_a+c} = e^c e^{z_j}/(e^c \sum_a e^{z_a}) = \mathrm{softmax}(z)_j$. Tomando $c = -z^\star$ y aplicando $\ell = -\ln \mathrm{softmax}(z)_y$ se obtiene la fórmula, en la que $z_a - z^\star \le 0$. ∎

La fórmula estable evita el desbordamiento: en precisión simple el mayor número representable es unos $3{,}4 \cdot 10^{38} \approx e^{88{,}7}$, de modo que e^{1000} daría infinito y el cociente, un valor indefinido. Es la técnica *log-sum-exp* del módulo 11, que CS336 exige en su implementación de la entropía cruzada [11, §4.1].

Proposición 1.13 (Pérdida de la predicción uniforme). *Si todos los logits de una posición son iguales, la pérdida vale* $\ln K$ *para cualquier objetivo.*

Demostración. Si $z = c\mathbf{1}$, por la proposición 1.12 $\mathrm{softmax}(z) = \mathrm{softmax}(\mathbf{0}) = (1/K, \ldots, 1/K)$, y $\ell = -\ln(1/K) = \ln K$. ∎

Con $K = 46$, $\ln 46 = 3{,}829$ nats (5,52 bits). Una red recién inicializada con pesos pequeños produce logits casi iguales, y su pérdida inicial está cerca de ese valor; en la práctica del módulo el primer lote da 4,046. Una pérdida inicial muy superior a $\ln K$ indica una inicialización defectuosa: el modelo está muy seguro de respuestas equivocadas.

Lecturas recomendadas para este tema

- Stanford CS336 [11], tarea 1, §4.1 «Cross-entropy loss» (ecuaciones 16–18) y §5.1 «Data Loader».
- Zhang *et al.* [16], §9.3.2 «Perplexity» y §9.3.3 «Partitioning Sequences».
- Stanford CS224N [15], clase 7 «Pretraining (Scaling, Systems, Data)».
- Goodfellow, Bengio y Courville [4], §8.1.3 «Batch and Minibatch Algorithms».

Ejercicios del tema 1

Ejercicio 1.1 ⋆ ¿Por qué la ventana objetivo y es la ventana de entrada x desplazada una posición? ¿Cuántas predicciones contiene un lote de forma (B, T)? Particularice para $B = T = 32$.

Ejercicio 1.2 ⋆⋆ Con el texto «hola\n» y $T = 2$, escriba todas las ventanas (x, y) y cuente las predicciones. ¿Por qué no hay una ventana que empiece en el cuarto carácter?

Ejercicio 1.3 ⋆⋆ Un corpus tiene $n = 10$ tokens $u_1, \ldots, u_{10}$ y se usan ventanas de $T = 4$. (a) ¿Cuántas ventanas hay si cualquier inicio es válido? (b) Si en su lugar se corta el corpus en trozos consecutivos sin solape, empezando tras descartar los $k \in \{0, 1, 2, 3\}$ primeros tokens, ¿cuántos trozos se obtienen con cada k? Si se elige k al azar con probabilidad uniforme y después uno de los trozos de ese corte, también con probabilidad uniforme, ¿con qué probabilidad resulta elegida cada una de las ventanas de (a)? ¿Es uniforme este muestreo? ¿Qué procedimiento sí lo es? (c) ¿Qué problema aparecería si cada ejemplo tuviera que ser una frase completa, y cómo lo resuelve el corpus concatenado?

Fuente: adaptado de Zhang *et al.* [16], §9.3, ejercicios 4 y 5 (se fijan n y T y se pide el recuento explícito).

Ejercicio 1.4 ⋆⋆ (a) Calcule $\ell(o, y)$ para $o = (1000, 1001, 999)$ con objetivo el primer componente, aplicando la proposición 1.12. ¿Qué ocurriría al calcular e^{1000} en precisión simple? (b) Una secuencia de 4 tokens tiene pérdidas (0,5; 1,2; 0,1; 2,0). Calcule su perplejidad.

Fuente: adaptado de Stanford CS336 [11], tarea 1, §4.1, problema *cross_entropy* y ecuación (18) (datos numéricos añadidos).

Ejercicio 1.5 ⋆⋆ Un lote tiene pérdida media 0,7 nats por carácter. Calcule su perplejidad y su pérdida en bits por carácter, y compruebe que la perplejidad no depende de la base.

Ejercicio 1.6 ⋆⋆⋆ Demuestre que, si la capa de salida del modelo, $z = Wh + b$, se inicializa con $W = 0$ y $b = 0$, la pérdida inicial de cualquier lote es exactamente $\ln K$, sean cuales sean los demás parámetros. Demuestre además que, en ese punto, el gradiente de la pérdida respecto de b es $\frac{1}{K}\mathbf{1} - \hat{\pi}$, donde $\hat{\pi}$ es la frecuencia empírica de cada token entre los objetivos del lote, y explique qué aprende el sesgo en el primer paso de descenso de gradiente.

Tema 2

La arquitectura mini-GPT

El mini-GPT es el bloque Transformer del módulo 14 apilado L veces entre una capa de entrada, que convierte tokens y posiciones en vectores, y una capa de salida, que devuelve un logit por token del vocabulario. Ninguna pieza es nueva; este tema fija la definición completa del modelo, demuestra la propiedad de causalidad que justifica entrenar las T predicciones de una ventana con una sola pasada y cuenta sus parámetros. La arquitectura es la de GPT-2 [10, §2.3], construida sobre el decoder de Vaswani *et al.* [13]; se sigue también el §3 de la tarea 1 de CS336 [11], la clase 5 de CS224N [15] y el §11.9.3 de [16].

2.1 Definición del modelo

Se recuerdan del módulo 11 la *normalización de capa* de un vector $v \in \mathbb{R}^d$,

$$\mathrm{LN}(v) = \gamma \odot \frac{v - \mu(v)\mathbf{1}}{\sqrt{\sigma^2(v) + \varepsilon}} + \beta, \qquad \mu(v) = \frac{1}{d}\sum_i v_i, \quad \sigma^2(v) = \frac{1}{d}\sum_i \big(v_i - \mu(v)\big)^2,$$

con parámetros $\gamma, \beta \in \mathbb{R}^d$, y del módulo 14 el perceptrón por posición $\mathrm{MLP}(v) = W_2\,\mathrm{GELU}(W_1 v + b_1) + b_2$, con $W_1 \in \mathbb{R}^{4d\times d}$ y $W_2 \in \mathbb{R}^{d\times 4d}$.

Definición 2.1 (Atención causal multicabeza). Sea $H \in \mathbb{R}^{T\times d}$ la matriz cuyas filas son las representaciones de las T posiciones, y sea n_h un divisor de d, con $d_k = d/n_h$. Se calculan $\boldsymbol{Q} = HW_Q$, $\boldsymbol{K} = HW_K$ y $\boldsymbol{V} = HW_V$, con $W_Q, W_K, W_V \in \mathbb{R}^{d\times d}$, y se parten por columnas en n_h bloques $\boldsymbol{Q}_r, \boldsymbol{K}_r, \boldsymbol{V}_r \in \mathbb{R}^{T\times d_k}$. La *máscara causal* es la matriz $\boldsymbol{M} \in \{0, -\infty\}^{T\times T}$ con $\boldsymbol{M}_{tj} = 0$ si $j \le t$ y $\boldsymbol{M}_{tj} = -\infty$ si $j > t$. Cada cabeza calcula

$$\mathrm{head}_r = \mathrm{softmax}\Big(\frac{\boldsymbol{Q}_r \boldsymbol{K}_r^\top}{\sqrt{d_k}} + \boldsymbol{M}\Big)\boldsymbol{V}_r, \qquad r = 1, \dots, n_h,$$

con la softmax aplicada por filas y el convenio $e^{-\infty} = 0$, y la salida es $\mathrm{MHA}(H) = [\mathrm{head}_1 \ \cdots \ \mathrm{head}_{n_h}]\, W_O + \mathbf{1} b_O^\top$, con $W_O \in \mathbb{R}^{d\times d}$ y $b_O \in \mathbb{R}^d$.

En la implementación, las tres proyecciones se agrupan en una sola matriz $d \times 3d$ sin sesgo, como sugiere CS336 [11, §3.4]; el número de parámetros es el mismo.

Definición 2.2 (Mini-GPT). Sean K, T, d, L y n_h enteros positivos con $n_h \mid d$. El *mini-GPT* con estos hiperparámetros transforma una ventana $x_{1:T} \in \mathcal{V}^T$ en logits $z_{1:T} \in \mathbb{R}^{T\times K}$ mediante:

(1) *entrada*: $h_t^0 = E_{x_t} + P_t$, donde $E \in \mathbb{R}^{K\times d}$ es la tabla de embeddings de token y $P \in \mathbb{R}^{T\times d}$ la de embeddings de posición, ambas aprendidas;

(2) *bloques*: para $\ell = 1, \dots, L$, con $H^{\ell-1}$ la matriz de filas $h_t^{\ell-1}$,

$$A^\ell = H^{\ell-1} + \mathrm{MHA}_\ell\big(\mathrm{LN}_{\ell,1}(H^{\ell-1})\big), \qquad H^\ell = A^\ell + \mathrm{MLP}_\ell\big(\mathrm{LN}_{\ell,2}(A^\ell)\big),$$

donde LN y MLP se aplican a cada fila por separado;

(3) *salida*: $z_t = W_{\mathrm{s}}\,\mathrm{LN}_{\mathrm{f}}(h_t^L) + b_{\mathrm{s}}$, con $W_{\mathrm{s}} \in \mathbb{R}^{K\times d}$ y $b_{\mathrm{s}} \in \mathbb{R}^K$.

La distribución predicha en la posición t es $p_t = \mathrm{softmax}(z_t)$.

La forma $x \leftarrow x + f(\mathrm{LN}(x))$ se llama *pre-norm*: la normalización se aplica a la entrada de cada subcapa y el flujo residual queda sin normalizar, lo que estabiliza el entrenamiento de redes profundas (módulo 11). GPT-2 introdujo exactamente estos dos cambios respecto de GPT: la normalización en la entrada de cada subbloque y una normalización final tras el último bloque [10, §2.3]. La figura 2.1 resume el flujo de datos. Suele decirse que la atención es la etapa en la que las posiciones se *comunican* y el MLP aquella en la que cada posición *procesa* lo recibido.

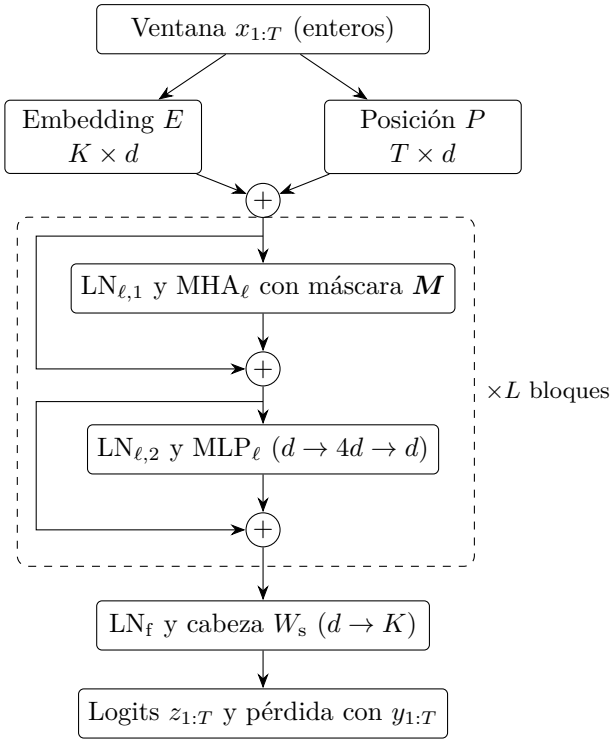


Figura 2.1: Flujo de datos del mini-GPT de la definición 2.2. Las flechas de la izquierda son las conexiones residuales; cada subcapa normaliza su entrada (pre-norm).

2.2 La máscara causal

Lema 2.3. *Sea $S \in \mathbb{R}^{T\times T}$ una matriz de puntuaciones. La fila t de* $\mathrm{softmax}(S + \boldsymbol{M})$ *vale*

$$a_{tj} = \begin{cases} \dfrac{e^{S_{tj}}}{\sum_{j'\le t} e^{S_{tj'}}} & j \le t, \\ 0 & j > t, \end{cases}$$

es decir, es la softmax de $(S_{t1}, \ldots, S_{tt})$ completada con ceros. En particular, cada fila suma 1 y la fila 1 es $(1, 0, \ldots, 0)$.

Demostración. Con el convenio $e^{-\infty} = 0$, los términos con $j > t$ no contribuyen al denominador y su numerador es nulo. El denominador contiene al menos el término $j = 1$, que es positivo, luego la expresión está bien definida. ■

Ejemplo 2.4. Con $T = 3$ y puntuaciones por filas $(1, 0, 2)$, $(2, 1, 0)$ y $(0, 1, 1)$, la máscara anula las entradas por encima de la diagonal y la softmax por filas da

$$\begin{pmatrix} 1 & 0 & 0 \\ 0{,}731 & 0{,}269 & 0 \\ 0{,}155 & 0{,}422 & 0{,}422 \end{pmatrix}.$$

La segunda fila es $\mathrm{softmax}(2, 1)$; la tercera, $\mathrm{softmax}(0, 1, 1) = (1, e, e)/(1 + 2e)$. La primera posición solo se atiende a sí misma.

Teorema 2.5 (Causalidad). *En el mini-GPT, para cada $t \in \{1, \dots, T\}$, los logits z_t dependen solo de $x_{1:t}$: si dos ventanas x y x' coinciden en las t primeras posiciones, entonces $z_t(x) = z_t(x')$.*

Demostración. Se demuestra por inducción sobre ℓ la afirmación $\mathcal{P}(\ell)$: *para todo s, la fila h_s^ℓ depende solo de $x_{1:s}$.*

Base. $h_s^0 = E_{x_s} + P_s$ depende solo de x_s.

Paso. Supóngase $\mathcal{P}(\ell - 1)$. La normalización actúa fila a fila, de modo que la fila s de $\mathrm{LN}_{\ell,1}(H^{\ell-1})$ depende solo de $h_s^{\ell-1}$ y, por la hipótesis, de $x_{1:s}$. Lo mismo ocurre con las filas s de $\boldsymbol{Q}_r$, $\boldsymbol{K}_r$ y $\boldsymbol{V}_r$, que son productos de esa fila por matrices fijas. Por el lema 2.3, la fila t de head_r es $\sum_{j \le t} a_{tj} (\boldsymbol{V}_r)_j$, con pesos a_{tj} que son la softmax de $(\boldsymbol{Q}_r)_t \cdot (\boldsymbol{K}_r)_j / \sqrt{d_k}$ para $j \le t$. Todas estas cantidades proceden de filas $j \le t$, que dependen de $x_{1:j} \subseteq x_{1:t}$. La concatenación, el producto por W_O y la suma residual con $h_t^{\ell-1}$ conservan la propiedad, luego la fila t de A^ℓ depende solo de $x_{1:t}$. El MLP, la segunda normalización y la segunda suma residual actúan fila a fila, de modo que h_t^ℓ depende solo de $x_{1:t}$. Esto prueba $\mathcal{P}(\ell)$.

Por inducción se cumple $\mathcal{P}(L)$, y z_t se obtiene de h_t^L aplicando operaciones fila a fila. ■

Corolario 2.6. *Una sola evaluación del mini-GPT sobre la ventana $x_{1:T}$ produce, en cada posición t, los mismos logits que se obtendrían evaluándolo sobre el prefijo $x_{1:t}$. Por tanto, una pasada calcula las T distribuciones $P_\theta(\cdot \mid x_{1:t})$ de la definición 1.8, cada una con su contexto correcto.*

Demostración. Evaluar el modelo sobre el prefijo equivale a evaluarlo sobre cualquier ventana que lo extienda, y por el teorema 2.5 el valor de z_t no depende de cómo se complete. ■

Observación 2.7 (Fuga del futuro). Sin la máscara, la posición t puede atender a $x_{t+1} = y_t$. Una cabeza que copie el token siguiente lleva la pérdida a casi cero sin aprender a predecir nada, y el modelo falla al generar, cuando el futuro no existe. Una pérdida de validación próxima a cero en los primeros cientos de pasos es el síntoma característico (cuadro 4.2). La implementación original pone a $-\infty$ las entradas de la softmax correspondientes a conexiones ilegales [13, §3.2.3].

2.3 Recuento de parámetros

Proposición 2.8 (Parámetros del mini-GPT). *Con proyecciones W_Q, W_K, W_V sin sesgo, sesgos en W_O, en el MLP y en la cabeza, y sin compartir E con W_{s}, el mini-GPT de la definición 2.2 tiene*

$$N = \underbrace{Kd}_{E} + \underbrace{Td}_{P} + L\,(12d^2 + 10d) + \underbrace{2d}_{\mathrm{LN_f}} + \underbrace{dK + K}_{\textit{cabeza}}$$

parámetros. El número de cabezas n_h no interviene.

Demostración. En cada bloque: las proyecciones Q, K, V aportan $3d^2$ y la de salida $d^2 + d$; el MLP, $(4d \cdot d + 4d) + (d \cdot 4d + d) = 8d^2 + 5d$; las dos normalizaciones, $2 \cdot 2d = 4d$. La suma es $12d^2 + 10d$. Partir W_Q, W_K, W_V en n_h bloques de columnas no cambia su tamaño, luego n_h no interviene. Se añaden las dos tablas de entrada, la normalización final y la cabeza. ∎

Cuadro 2.1: Parámetros del mini-GPT de la práctica: $K = 46$, $T = 32$, $d = 64$, $L = 2$, $n_h = 4$.

Pieza	Fórmula	Parámetros
Embedding de token	Kd	2 944
Embedding de posición	Td	2 048
Atención (por bloque)	$3d^2 + d^2 + d$	16 448
MLP (por bloque)	$8d^2 + 5d$	33 088
Dos normalizaciones (por bloque)	$4d$	256
Bloque completo	$12d^2 + 10d$	49 792
Dos bloques	$2\,(12d^2 + 10d)$	99 584
Normalización final	$2d$	128
Cabeza	$dK + K$	2 990
Total		107 694

El 92 % de los parámetros está en los bloques ($99\,584/107\,694 = 0{,}925$). El término $12d^2$ crece con el cuadrado de la anchura, mientras que las tablas crecen linealmente en d; en los modelos grandes la proporción de los bloques es aún mayor, salvo que el vocabulario sea muy grande (ejercicio 2.5). Es la misma cuenta que el módulo 14 hace para un modelo de $7 \cdot 10^9$ parámetros.

Observación 2.9 (Pesos atados). Vaswani *et al.* comparten la matriz de embeddings con la transformación lineal previa a la softmax [13, §3.4], de modo que $W_{\mathrm{s}} = E$ y se ahorran Kd parámetros. El mini-GPT de la práctica no los ata; con atadura tendría $107\,694 - 2\,944 = 104\,750$ parámetros.

2.4 Del mini-GPT a los modelos abiertos

Los modelos abiertos más citados usan la misma estructura con algunas piezas sustituidas. LLaMA normaliza la entrada de cada subcapa con RMSNorm, sustituye la GELU por SwiGLU con dimensión oculta $\frac{2}{3} \cdot 4d$ y elimina los embeddings de posición absolutos en favor de las rotaciones RoPE en cada capa [12, §2.2]; CS336 adopta esas mismas piezas [11, §3]. GPT-2, además de la normalización en la entrada de cada subbloque, escala por $1/\sqrt{N_{\mathrm{r}}}$ al inicializar los pesos de las capas residuales, con N_{r} el número de capas residuales, usa un vocabulario de 50 257 tokens y un contexto de 1024 [10, §2.3]. El cuadro 2.2 resume las diferencias.

Cuadro 2.2: Componentes del mini-GPT, de GPT-2 y de LLaMA, según los artículos citados y la implementación de GPT-2 de nanoGPT [6].

Componente	Mini-GPT	GPT-2 [10]	LLaMA [12]
Posiciones	tabla aprendida $T \times d$	tabla aprendida	RoPE en cada capa
Normalización	LayerNorm pre-norm	LayerNorm pre-norm y final	RMSNorm pre-norm
MLP	GELU, $d \to 4d \to d$	GELU, $d \to 4d \to d$	SwiGLU, dimensión $\frac{2}{3} \cdot 4d$
Tokenizador	caracteres, $K = 46$	BPE de bytes, $K = 50\,257$	BPE (SentencePiece)

Sobre los modelos comerciales, la información pública es limitada: el informe técnico de Gemini indica que sus modelos se construyen sobre decoders Transformer [3, §2]; xAI publicó en 2024 los pesos y el código de Grok-1, un Transformer de 64 capas con mezcla de 8 expertos y RoPE [14], y Anthropic no ha publicado la arquitectura de Claude.

Lecturas recomendadas para este tema

- Radford *et al.* [10], §2.3 «Model».
- Vaswani *et al.* [13], §3.1–3.4, en particular §3.2.3 sobre la máscara y §3.4 sobre los pesos compartidos.
- Stanford CS336 [11], tarea 1, §3.4 «Pre-Norm Transformer Block» y §3.5 «The Full Transformer LM».
- Stanford CS224N [15], clase 5 «Transformers».
- Zhang *et al.* [16], §11.9.3 «Decoder-Only».
- Karpathy [6], archivo `model.py` del repositorio nanoGPT.

Ejercicios del tema 2

Ejercicio 2.1 ⋆ ¿Qué ocurre si se quita la máscara causal durante el entrenamiento? ¿Y si se entrena con máscara y se quita solo al generar? Distinga lo que ocurre en el primer bloque y en los siguientes.

Ejercicio 2.2 ⋆⋆ Una cabeza de atención con $T = 3$ tiene puntuaciones escaladas por filas $(0, 1, 3)$, $(1, 1, 2)$ y $(2, 0, 1)$ y valores $v_1 = (1, 0)$, $v_2 = (0, 1)$, $v_3 = (1, 1)$. Calcule la matriz de pesos tras la máscara causal y la salida de la cabeza en cada posición. ¿Qué entradas de la matriz de puntuaciones no influyen en el resultado?

Ejercicio 2.3 ⋆⋆ Cuente los parámetros del mini-GPT con $K = 46$, $T = 32$, $d = 32$ y $L = 2$, y compare la parte de los bloques con la del modelo del cuadro 2.1. ¿Cuántos parámetros tendría con pesos atados?

Ejercicio 2.4 ⋆⋆ Se compara el mini-GPT base ($d = 64$, $n_h = 4$) con una variante de una sola cabeza ($n_h = 1$). (a) ¿Cuántos parámetros tiene cada uno? (b) ¿Cuál es la dimensión d_k de cada cabeza y cuántas matrices de pesos de atención $T \times T$ calcula cada bloque en cada caso? (c) En la práctica, la variante de una cabeza termina con pérdida de validación 0,336 frente a 0,324. Explique por qué la diferencia no puede atribuirse a la capacidad medida en número de parámetros.

Ejercicio 2.5 ⋆⋆ GPT-2 *small* tiene $K = 50\,257$, $T = 1024$, $d = 768$ y $L = 12$, usa sesgo también en las proyecciones Q, K, V y comparte la matriz de embeddings con la cabeza, que no tiene sesgo. (a) Adapte la proposición 2.8 y calcule el número de parámetros. (b) ¿Cuánta memoria ocupan en precisión simple? (c) ¿Qué fracción de los parámetros está en los bloques y cuál en las tablas de embeddings? Compárela con la del mini-GPT.

Fuente: adaptado de Stanford CS336 [11], tarea 1, §3.5, problema *transformer_accounting*, apartados (a) y (d) (se sustituye la arquitectura de la tarea por la de GPT-2 y el tamaño XL por el *small*).

Ejercicio 2.6 ⋆⋆⋆ Considérese el mini-GPT sin embeddings de posición ($P = 0$). Sea σ una permutación de $\{1, \dots, T\}$ y x^σ la ventana con $x^\sigma_t = x_{\sigma(t)}$. (a) Demuestre que, si se suprime la máscara ($\boldsymbol{M} = 0$), el modelo es equivariante: $z_t(x^\sigma) = z_{\sigma(t)}(x)$ para todo t y cualquier L. (b) Demuestre que, con máscara y $L = 1$, los logits de la última posición son invariantes frente a cualquier permutación que fije T. (c) Explique por qué el argumento de (b) no se extiende a $L \geq 2$ y qué consecuencia tiene sobre la posibilidad de entrenar un decoder sin codificación posicional.

Tema 3

Optimización: AdamW, calendario y recorte

Entrenar el mini-GPT consiste en repetir un paso muy breve: tomar un lote, calcular la pérdida, retropropagar y actualizar los parámetros. Lo que separa un entrenamiento estable de uno que diverge son tres piezas que rodean ese paso: el optimizador AdamW, el calendario de la tasa de aprendizaje y el recorte del gradiente. Este tema las define tal como las especifica la tarea 1 de CS336 [11, §4.2–4.5], demuestra sus propiedades principales y estima el coste del entrenamiento. AdamW y el calendario coseno se estudiaron en el módulo 7; aquí se usan en su forma de implementación, siguiendo a Loshchilov y Hutter [9, 8] y la receta documentada por LLaMA [12, §2.3].

3.1 El paso de entrenamiento

Definición 3.1 (Paso de entrenamiento). Dados los parámetros θ_s y el estado del optimizador en el paso $s = 0, 1, \ldots, S-1$, un *paso de entrenamiento* consiste en:

1. fijar la tasa η_s según el calendario;
2. extraer un lote (X, Y) (definición 1.6);
3. calcular los logits y la pérdida $\mathcal{L}(\theta_s)$ (definición 1.8);
4. poner a cero los gradientes acumulados y calcular $g = \nabla_\theta \mathcal{L}(\theta_s)$ por retropropagación;
5. recortar g por norma global (definición 3.9);
6. aplicar un paso de AdamW con tasa η_s (definición 3.2);
7. cada cierto número de pasos, evaluar la pérdida de entrenamiento y de validación y guardar un punto de control si la validación mejora (tema 4).

El orden importa. El recorte actúa sobre el gradiente ya calculado y antes de que el optimizador lo incorpore a sus momentos; si se aplicara después, un gradiente anómalo ya habría contaminado m y v. En PyTorch, el paso 4 son las llamadas `zero_grad` y `backward`, y el paso 6 es `step` [11, §4.2.1].

3.2 AdamW

Definición 3.2 (AdamW, forma de implementación). Sean $\beta_1, \beta_2 \in [0,1)$, $\lambda \geq 0$ y $\varepsilon > 0$, y sean $m_0 = v_0 = 0$. En la iteración $t = 1, 2, \ldots$, con tasa α y gradiente g_t, AdamW realiza

$$\alpha_t = \alpha \frac{\sqrt{1-\beta_2^t}}{1-\beta_1^t}, \qquad \theta \leftarrow \theta - \alpha\lambda\theta,$$
$$m_t = \beta_1 m_{t-1} + (1-\beta_1) g_t, \qquad v_t = \beta_2 v_{t-1} + (1-\beta_2) g_t^2,$$
$$\theta \leftarrow \theta - \alpha_t \frac{m_t}{\sqrt{v_t} + \varepsilon},$$

donde el cuadrado, la raíz y el cociente se aplican componente a componente.

Es el algoritmo 1 de CS336 [11, §4.3], que reproduce el algoritmo 2 de Loshchilov y Hutter [9]. La corrección de sesgo de los momentos del módulo 7 aparece aquí agrupada en α_t; la diferencia con la forma $\hat{m}_t/(\sqrt{\hat{v}_t}+\varepsilon)$ es solo la escala efectiva de ε. El término $-\alpha\lambda\theta$ es el *decaimiento de pesos desacoplado*: no pasa por los momentos y encoge todos los pesos en la misma proporción.

Proposición 3.3 (Encogimiento por decaimiento). *Si un parámetro recibe gradiente nulo durante los pasos $s = 0, \ldots, S-1$, con tasas η_s y $\lambda\eta_s < 1$, su valor final es*

$$\theta_S = \theta_0 \prod_{s=0}^{S-1} (1 - \lambda\eta_s) \;\leq\; \theta_0 \exp\Big(-\lambda \sum_{s=0}^{S-1} \eta_s\Big) \qquad (\theta_0 \geq 0).$$

Demostración. Con $g = 0$ se tiene $m_t = v_t = 0$ en todo paso, y la actualización se reduce a $\theta \leftarrow (1-\lambda\eta_s)\theta$. Iterando se obtiene el producto. La cota se sigue de $1 - u \leq e^{-u}$ para todo $u \in \mathbb{R}$, aplicada a cada factor positivo. ∎

Ejemplo 3.4. Con $\lambda = 0{,}1$ y el calendario del ejemplo 3.7, la suma de las tasas de los 800 pasos es $\sum_s \eta_s = 1{,}308$, de modo que un peso sin gradiente se multiplica por $\prod_s (1-\lambda\eta_s) = 0{,}877$, prácticamente igual a $e^{-0{,}1308} = 0{,}877$. Con la tasa constante $3 \cdot 10^{-3}$, el factor sería $0{,}9997^{800} = 0{,}787$.

Los momentos m y v forman el *estado* del optimizador: dos números por parámetro, además del contador t. Por eso el estado ocupa el doble que el modelo y debe guardarse en los puntos de control (tema 4). LLaMA usa $\beta_1 = 0{,}9$, $\beta_2 = 0{,}95$ y $\lambda = 0{,}1$ [12, §2.3]; CS336 señala que (0,9; 0,999) es la elección típica y (0,9; 0,95) la habitual en modelos de lenguaje grandes [11, §4.3].

3.3 Calentamiento lineal y decaimiento coseno

Definición 3.5 (Calendario del módulo). Sean $0 \leq \eta_{\min} < \eta_{\max}$, $W \geq 1$ pasos de calentamiento y $S > W$ pasos totales. Para $s = 0, \ldots, S-1$,

$$\eta_s = \begin{cases} \eta_{\max} \dfrac{s+1}{W}, & s < W, \\[2ex] \eta_{\min} + \tfrac{1}{2}(\eta_{\max} - \eta_{\min})\Big(1 + \cos\dfrac{\pi(s-W)}{S-W}\Big), & s \geq W. \end{cases}$$

El tramo coseno es la ecuación (5) de SGDR [8, §3] con un único ciclo. La práctica usa $\eta_{\min} = \eta_{\max}/10$, como LLaMA [12, §2.3] y la configuración de ejemplo de nanoGPT [6].

Proposición 3.6 (Propiedades del calendario). *En el calendario de la definición 3.5:*

(i) η_s *crece estrictamente en* $\{0, \dots, W-1\}$*, con* $\eta_0 = \eta_{\max}/W$ *y* $\eta_{W-1} = \eta_W = \eta_{\max}$*;*

(ii) η_s *decrece estrictamente en* $\{W, \dots, S-1\}$*, y la expresión del tramo coseno vale* $\eta_{\min}$ *en* $s = S$*;*

(iii) en el punto medio del tramo coseno, $s = W + (S-W)/2$*, vale* $(\eta_{\max} + \eta_{\min})/2$*;*

(iv) para todo $u \in [0, S-W]$*,* $\eta(W+u) + \eta(S-u) = \eta_{\max} + \eta_{\min}$*, donde* $\eta(\cdot)$ *denota la expresión del tramo coseno evaluada en un real.*

Demostración. (i) En el tramo lineal, η_s es una función afín de pendiente $\eta_{\max}/W > 0$; en $s = W-1$ vale $\eta_{\max}W/W$, y en $s = W$ el coseno vale $\cos 0 = 1$ y $\eta_W = \eta_{\min} + (\eta_{\max} - \eta_{\min}) = \eta_{\max}$. (ii) El argumento $\pi(s-W)/(S-W)$ recorre $[0, \pi)$, intervalo en el que el coseno decrece estrictamente; en $s = S$ el argumento es π y $1 + \cos\pi = 0$. (iii) En el punto medio el argumento es $\pi/2$ y $\cos(\pi/2) = 0$. (iv) Con $\phi = \pi u/(S-W)$, los argumentos de $\eta(W+u)$ y $\eta(S-u)$ son ϕ y $\pi - \phi$, y $\cos(\pi - \phi) = -\cos\phi$; al sumar, los cosenos se cancelan y queda $2\eta_{\min} + (\eta_{\max} - \eta_{\min})$. ∎

Ejemplo 3.7. Con $S = 800$, $W = 100$, $\eta_{\max} = 3 \cdot 10^{-3}$ y $\eta_{\min} = 3 \cdot 10^{-4}$:

Paso s	Fase	η_s
0	calentamiento	$3 \cdot 10^{-5}$
49	calentamiento	$1{,}5 \cdot 10^{-3}$
100	pico	$3 \cdot 10^{-3}$
275	un cuarto del coseno	$2{,}605 \cdot 10^{-3}$
450	mitad del coseno	$1{,}65 \cdot 10^{-3}$
625	tres cuartos del coseno	$6{,}954 \cdot 10^{-4}$
799	último paso	$3{,}000 \cdot 10^{-4}$

Los pasos 275 y 625 son simétricos respecto del punto medio: $2{,}605 \cdot 10^{-3} + 0{,}695 \cdot 10^{-3} = 3{,}3 \cdot 10^{-3} = \eta_{\max} + \eta_{\min}$, como afirma la proposición 3.6 (iv).

Observación 3.8 (El convenio de CS336). CS336 define el calentamiento como $\alpha_t = (t/T_w)\,\alpha_{\max}$ para $t < T_w$, el coseno entre T_w y T_c y una tasa constante $\alpha_{\min}$ para $t > T_c$ [11, §4.4]. Con ese convenio, $\alpha_0 = 0$ (el primer paso no mueve los pesos, salvo por el decaimiento, que también se multiplica por α) y el pico se alcanza en $t = T_w$; con el del módulo, la tasa empieza en $\eta_{\max}/W$ y llega al pico en $s = W - 1$. Ambas formas difieren en un paso y son equivalentes en la práctica.

La justificación de ambas fases se dio en el módulo 7. Durante el calentamiento, los momentos de Adam se han estimado con muy pocos gradientes, y estos son grandes e irregulares; una tasa grande desde el primer paso puede llevar a una región de la que el entrenamiento no se recupera. Al final, una tasa pequeña reduce el exceso de pérdida debido al ruido del gradiente estocástico. LLaMA documenta 2000 pasos de calentamiento y decaimiento coseno hasta el 10 % de la tasa máxima [12, §2.3].

3.4 Recorte del gradiente por norma global

Definición 3.9 (Recorte por norma global). Sea $g = (g^{(1)}, \dots, g^{(P)})$ el gradiente de todos los tensores de parámetros, considerado como un único vector, y $\|g\|_2 = \left(\sum_q \|g^{(q)}\|_2^2\right)^{1/2}$ su

norma global. Dados $c > 0$ y $\varepsilon > 0$ pequeño,

$$\operatorname{clip}_c(g) = \begin{cases} g, & \|g\|_2 < c, \\ \dfrac{c}{\|g\|_2 + \varepsilon}\, g, & \|g\|_2 \geq c. \end{cases}$$

Es la definición de CS336, con $\varepsilon = 10^{-6}$ [11, §4.5]; LLaMA usa $c = 1$ [12, §2.3], como la práctica del módulo. El recorte protege de los gradientes anómalos que aparecen cuando la superficie de pérdida tiene acantilados [4, §8.2.4], y que de otro modo dañarían durante muchos pasos las medias móviles de Adam.

Proposición 3.10 (Propiedades del recorte). *Para todo g:*

(i) $\operatorname{clip}_c(g)$ *es un múltiplo positivo de g, luego tiene su misma dirección y sentido;*

(ii) $\|\operatorname{clip}_c(g)\|_2 < c$ *si* $\|g\|_2 \geq c$, *y* $\operatorname{clip}_c(g) = g$ *en otro caso;*

(iii) recortar cada tensor por separado con el mismo umbral produce, en general, un vector que no es múltiplo de g.

Demostración. (i) El factor $c/(\|g\|_2 + \varepsilon)$ es positivo. (ii) Si $\|g\|_2 \geq c$, entonces $\|\operatorname{clip}_c(g)\|_2 = c\,\|g\|_2/(\|g\|_2 + \varepsilon) < c$. (iii) Basta un contraejemplo: con dos tensores $g^{(1)} = (3, 4)$ y $g^{(2)} = (12)$ y $c = 1$, el recorte por tensor (con $\varepsilon \to 0$) da (0,6; 0,8) y (1). El vector resultante (0,6; 0,8; 1) no es proporcional a $(3, 4, 12)$, porque los cocientes componente a componente son 0,2, 0,2 y 1/12. ∎

El recorte global no cambia la dirección de descenso; solo limita la longitud del paso. El recorte por tensor, en cambio, altera el peso relativo de cada parte del modelo en la actualización.

3.5 El coste del entrenamiento

Por la regla del módulo 14, entrenar un modelo de N parámetros con D tokens cuesta $C \approx 6ND$ operaciones de coma flotante (FLOP): $2ND$ en la pasada hacia delante y el doble en la retropropagación. CS336 usa la misma hipótesis sobre la retropropagación [11, §4.3, problema *adamw_accounting*].

Ejemplo 3.11. En la práctica, $N = 107\,694$ y $D = 800 \cdot 32 \cdot 32 = 819\,200$ tokens, luego $C \approx 6 \cdot 107\,694 \cdot 819\,200 = 5{,}29 \cdot 10^{11}$ FLOP. El entrenamiento dura unos 17 s en un núcleo de CPU libre, lo que corresponde a unos $3 \cdot 10^{10}$ FLOP/s efectivos. Con matrices tan pequeñas el procesador trabaja muy por debajo de su pico (módulo 17).

Lecturas recomendadas para este tema

- Stanford CS336 [11], tarea 1, §4.2 «The SGD Optimizer», §4.3 «AdamW», §4.4 «Learning rate scheduling» y §4.5 «Gradient clipping».
- Loshchilov y Hutter [9], algoritmo 2; y [8], §3, ecuación (5).
- Touvron *et al.* [12], §2.3 «Optimizer».
- Goodfellow, Bengio y Courville [4], §8.2.4 «Cliffs and Exploding Gradients» y §8.5.3 «Adam».
- Zhang *et al.* [16], §9.5.3 «Gradient Clipping».

Ejercicios del tema 3

Ejercicio 3.1 ⋆ En el paso de entrenamiento de la definición 3.1, explique por qué el recorte se aplica después de la retropropagación y antes del paso del optimizador, y por qué se usa la norma global y no la de cada tensor. ¿Qué problema resuelve el calentamiento y cuál el decaimiento coseno?

Ejercicio 3.2 ⋆⋆ Con el calendario de la definición 3.5, $W = 200$, $S = 2000$, $\eta_{\max} = 10^{-3}$ y $\eta_{\min} = 10^{-4}$, calcule η_9 y η_{1000}. Compruebe el segundo valor con la proposición 3.6.

Ejercicio 3.3 ⋆⋆ Con el convenio de la observación 3.8, $\alpha_{\max} = 10^{-3}$, $\alpha_{\min} = 10^{-4}$, $T_w = 100$ y $T_c = 1000$, calcule α_t en $t = 0$, 50, 100, 325, 550, 775, 1000 y 1200. ¿En qué se diferencia del calendario del módulo?

Fuente: adaptado de Stanford CS336 [11], tarea 1, §4.4, problema *learning_rate_schedule* (se pide el cálculo a mano en puntos concretos).

Ejercicio 3.4 ⋆⋆ Con $c = 1$ y $\varepsilon = 10^{-6}$: (a) recorte $g = (3, 4, 0)$; (b) recorte $g = (0{,}3;\ 0{,}4;\ 0)$; (c) un modelo tiene dos tensores con gradientes $(3, 4)$ y (12). Recorte con la norma global y compare con el recorte por tensor: ¿qué fracción de la norma corresponde al primer tensor en cada caso?

Fuente: adaptado de Stanford CS336 [11], tarea 1, §4.5, problema *gradient_clipping* (datos numéricos añadidos).

Ejercicio 3.5 ⋆⋆ El SGD con tasa decreciente de CS336 actualiza

$$w \leftarrow w - \frac{\alpha}{\sqrt{t+1}}\,\nabla L(w), \qquad t = 0, 1, 2, \ldots$$

Se minimiza $L(w) = \frac{1}{100}\sum_{i=1}^{100} w_i^2$. (a) Demuestre que cada paso multiplica todos los pesos por $f_t = 1 - \alpha/(50\sqrt{t+1})$. (b) Para $\alpha = 1$, 10, 100 y 1000, calcule f_0, f_1, f_2 y el cociente L_3/L_0. ¿Qué tasas hacen decrecer la pérdida y cuál la hace divergir?

Fuente: adaptado de Stanford CS336 [11], tarea 1, §4.2, problema *learning_rate_tuning* (se sustituye la simulación por el cálculo exacto de tres pasos).

Ejercicio 3.6 ⋆⋆ Estime los FLOP de entrenar el mini-GPT de 107 694 parámetros durante 2000 pasos con $B = T = 32$. Con el rendimiento efectivo del ejemplo 3.11, ¿cuánto duraría?

Ejercicio 3.7 ⋆⋆⋆ Sea $\mathcal{B}_c = \{u : \|u\|_2 \le c\}$ y sea $\Pi_c(g)$ la proyección euclídea de g sobre $\mathcal{B}_c$, es decir, el punto de $\mathcal{B}_c$ más próximo a g. Demuestre que $\Pi_c(g) = g$ si $\|g\|_2 \le c$ y $\Pi_c(g) = c\,g/\|g\|_2$ en otro caso, y concluya que el recorte por norma global con $\varepsilon = 0$ es la proyección sobre la bola de radio c. ¿Es el recorte por tensor la proyección sobre algún conjunto? Descríbalo.

Tema 4

Evaluación, puntos de control y curvas de pérdida

La pérdida de entrenamiento de un lote es una cantidad ruidosa que, además, no mide lo que interesa: cómo se comporta el modelo con texto que no ha visto. Este tema define el estimador de la pérdida que se usa para dibujar las curvas, describe qué debe contener un punto de control para poder reanudar el entrenamiento sin perturbarlo, demuestra que la entropía del proceso que genera el texto es una cota inferior de la pérdida esperada y presenta una guía de diagnóstico a partir de la forma de las curvas. Se sigue el §7.8 y el capítulo 11 de [4], los §5.2 y §7.2.3 de la tarea 1 de CS336 [11] y el capítulo 2 de Cover y Thomas [1].

4.1 Estimar la pérdida

Definición 4.1 (Pérdida estimada). Sean $(X^{(1)}, Y^{(1)}), \dots, (X^{(r)}, Y^{(r)})$ lotes extraídos de un conjunto de datos (entrenamiento o validación). La *pérdida estimada* del modelo de parámetros θ es

$$\hat{\mathcal{L}}(\theta) = \frac{1}{r} \sum_{q=1}^{r} \mathcal{L}_q(\theta),$$

donde $\mathcal{L}_q$ es la pérdida media del lote q (definición 1.8). En la práctica del módulo, $r = 20$ y los lotes se extraen con un generador de semilla fija, de modo que son los mismos en todas las evaluaciones.

Si los lotes se extraen de forma independiente, $\hat{\mathcal{L}}(\theta)$ es un estimador insesgado de la pérdida esperada y su varianza es la de un lote dividida por r. Usar siempre los mismos lotes tiene una ventaja adicional cuando se comparan dos puntos de la curva.

Proposición 4.2 (Lotes comunes). *Sean θ_a y θ_b dos estados del modelo, y sean $\hat{\mathcal{L}}_a$ y $\hat{\mathcal{L}}_b$ sus pérdidas estimadas, vistas como variables aleatorias a través de la elección de los lotes. Entonces*

$$\operatorname{Var}\big(\hat{\mathcal{L}}_a - \hat{\mathcal{L}}_b\big) = \operatorname{Var}(\hat{\mathcal{L}}_a) + \operatorname{Var}(\hat{\mathcal{L}}_b) - 2\operatorname{Cov}(\hat{\mathcal{L}}_a, \hat{\mathcal{L}}_b).$$

Si los dos estimadores usan lotes independientes, la covarianza es nula; si usan los mismos lotes y la covarianza es positiva, la varianza de la diferencia es menor.

Demostración. Es la fórmula de la varianza de una diferencia, $\operatorname{Var}(A - B) = \mathbb{E}[(A - B)^2] - (\mathbb{E}A - \mathbb{E}B)^2$, desarrollada. Con lotes independientes, $\hat{\mathcal{L}}_a$ y $\hat{\mathcal{L}}_b$ son funciones de variables independientes y su covarianza es nula. ∎

Dos estados próximos del modelo encuentran difíciles los mismos lotes, de modo que sus pérdidas sobre lotes comunes están positivamente correlacionadas; la diferencia entre dos

puntos de la curva refleja entonces sobre todo el cambio del modelo y no el azar del muestreo. Es la técnica de los *números aleatorios comunes* de la simulación estocástica.

Durante la evaluación se usan dos instrucciones de PyTorch: `model.eval()`, que desactiva los componentes que se comportan distinto al entrenar (como el *dropout*), y el contexto `torch.no_grad()`, que evita construir el grafo de cómputo y ahorra memoria y tiempo. Al terminar, `model.train()` devuelve el modelo al modo de entrenamiento.

4.2 Puntos de control

Definición 4.3 (Punto de control). Un *punto de control* es un registro que contiene todo lo necesario para reanudar el entrenamiento como si no se hubiera interrumpido: los parámetros θ, el estado del optimizador (en AdamW, los momentos m, v y el contador de pasos) y el número de paso s, que fija la tasa del calendario [11, §5.2]. Es habitual añadir la pérdida de validación y la configuración del modelo.

Si se reanuda sin el estado del optimizador, AdamW reinicia $m = v = 0$ y $t = 0$: el primer paso usa momentos estimados con un solo gradiente, sin la protección de un calentamiento, y la curva de pérdida da un salto. Si se reanuda sin el número de paso, el calendario vuelve a empezar.

Proposición 4.4 (Memoria del estado de entrenamiento). *En precisión simple (4 bytes por número), los parámetros, los gradientes y los dos momentos de AdamW de un modelo de N parámetros ocupan $16N$ bytes, sin contar las activaciones.*

Demostración. Cada uno de los cuatro tensores (parámetros, gradientes, m y v) tiene N componentes de 4 bytes: $4 \cdot 4N = 16N$. ∎

Para el mini-GPT son $107\,694 \cdot 16 = 1{,}72$ MB, de los que el modelo ocupa 0,43 MB. Para un modelo de $7 \cdot 10^9$ parámetros, la misma cuenta da 112 GB, más que la memoria de una GPU de 80 GB; por eso existen las técnicas de partición del estado del optimizador del módulo 17.

Observación 4.5 (Parada temprana). Guardar el punto de control solo cuando la pérdida de validación mejora y devolver al final el mejor es el algoritmo de *parada temprana* de Goodfellow, Bengio y Courville [4, §7.8], que actúa como regularizador: limita el número efectivo de pasos de optimización. Los grandes entrenamientos guardan además puntos de control frecuentes porque, con miles de GPU, las interrupciones son rutinarias: el informe de Llama 3 documenta 466 interrupciones en un periodo de 54 días de preentrenamiento [7, §3.3.4].

4.3 La cota inferior de la pérdida

Teorema 4.6 (La entropía como cota inferior). *Sean p y q distribuciones sobre $\mathcal{V}$ con $q(x) > 0$ siempre que $p(x) > 0$. Entonces*

$$H(p, q) = -\sum_{x} p(x) \ln q(x) = H(p) + D_{\mathrm{KL}}(p \| q) \;\geq\; H(p),$$

con igualdad si y solo si $q = p$.

Demostración. Sumando y restando $\sum_x p(x) \ln p(x)$,

$$-\sum_x p(x) \ln q(x) = -\sum_x p(x) \ln p(x) + \sum_x p(x) \ln \frac{p(x)}{q(x)} = H(p) + D_{\mathrm{KL}}(p\|q).$$

Por la desigualdad de Gibbs (módulo 6; [1, teorema 2.6.3]), $D_{\mathrm{KL}}(p\|q) \geq 0$ con igualdad si y solo si $p = q$. ■

Si p es la distribución real del token siguiente y q la del modelo, la pérdida esperada no puede bajar de $H(p)$. El ejercicio 4.6 extiende el resultado a distribuciones condicionadas al contexto, que es el caso de un modelo de lenguaje. En consecuencia, si la pérdida media de validación sale claramente por debajo de la entropía del proceso generador, hay una fuga de información. En texto humano no se conoce H, pero el mismo resultado explica por qué las curvas de los grandes modelos se aplanan: las leyes de escalado del módulo 16 incluyen un término constante que se interpreta como la entropía irreducible del texto.

Ejemplo 4.7 (Entropía del corpus sintético). Una línea de tipo «objeto» elige de forma uniforme e independiente uno de 6 objetos, 6 colores, 6 materiales y 6 usos; el resto de sus caracteres está determinado. Su entropía es $\ln 6^4 = 7{,}167$ nats, más $\ln 4 = 1{,}386$ nats por la elección del tipo de línea. Sumando la entropía de todas las líneas del corpus y dividiendo por el número de caracteres, el cuaderno obtiene $H \approx 0{,}125$ nats por carácter. El mini-GPT termina en 0,324: queda margen.

4.4 Lectura de las curvas de pérdida

Una curva de pérdida se interpreta respecto de tres referencias horizontales: $\ln K$, la pérdida de la predicción uniforme (proposición 1.13); la pérdida del modelo de bigramas, que es lo mínimo que hay que superar; y, cuando se conoce, la entropía H del texto, que no puede superarse (teorema 4.6). El cuadro 4.1 recoge la curva de la práctica.

Cuadro 4.1: Pérdidas del mini-GPT base de la práctica, en nats por carácter. Referencias: $\ln 46 = 3{,}829$; bigramas, 2,035; entropía del corpus, $\approx 0{,}125$.

Momento	Entrenamiento	Validación	Lectura
Paso 0	4,046 (un lote)	—	cerca de $\ln K$: predicción casi uniforme
Paso 100	2,017	2,046	alcanza el nivel del bigrama
Paso 200	1,040	1,154	aprende palabras y plantillas
Paso 400	0,370	0,425	usa el contexto largo
Paso 800	0,264	0,324	afinado final con tasa baja

La forma. Hay una caída rápida hasta el nivel del bigrama (pasos 0–100), en la que el modelo aprende las frecuencias de pares de caracteres, y una segunda bajada larga (100–500), en la que aprende a usar el contexto: qué plantilla está escribiendo y qué palabras han aparecido ya. La pendiente final se suaviza a medida que el coseno reduce la tasa.

La brecha. Al final, la diferencia entre validación y entrenamiento es 0,06 nats. Es un sobreajuste leve: el modelo ha memorizado algunas frases, lo que es esperable cuando cada token se

Cuadro 4.2: Síntomas frecuentes en las curvas de pérdida y su diagnóstico probable.

Síntoma	Diagnóstico probable	Qué probar
No baja de $\ln K$	error de programación: objetivos desalineados, tasa nula	sobreajustar un solo lote
Baja y después da NaN	tasa demasiado alta	bajar $\eta_{\max}$, alargar el calentamiento, recortar
Baja muy despacio	tasa demasiado baja	subir $\eta_{\max}$
Validación sube y entrenamiento baja	sobreajuste	más datos, decaimiento de pesos, parar antes
Pérdida casi nula en ambas	fuga del futuro	revisar la máscara causal (observación 2.7)

ve unas 57 veces. Si la pérdida de validación empezara a subir mientras la de entrenamiento baja, habría que parar, usar más datos o regularizar más.

La primera prueba de la tercera columna es la estrategia de depuración más recomendada: un modelo correcto debe poder llevar a casi cero la pérdida de un conjunto de datos diminuto, como un único lote; si no lo consigue, el fallo está en el software y no en la capacidad del modelo [4, §11.5], [11, §7.2.3]. La misma sección de [4] recomienda razonar con los errores de entrenamiento y de prueba: error bajo en entrenamiento y alto en prueba indica sobreajuste; error alto en ambos deja abiertas la falta de capacidad y un defecto del programa.

Lecturas recomendadas para este tema

- Goodfellow, Bengio y Courville [4], §7.8 «Early Stopping» (algoritmo de parada temprana) y §11.5 «Debugging Strategies».
- Stanford CS336 [11], tarea 1, §5.2 «Checkpointing» y §7.2.3 «Tips and tricks for debugging model architectures».
- Cover y Thomas [1], §2.6 «Jensen's inequality and its consequences».
- Llama Team [7], §3.3.4 «Reliability and Operational Challenges».

Ejercicios del tema 4

Ejercicio 4.1 ⋆ Explique por qué un punto de control debe incluir el estado del optimizador y el número de paso. ¿Qué se observaría en la curva de pérdida si se reanudara sin ellos?

Ejercicio 4.2 ⋆ ¿Por qué se evalúa siempre con los mismos lotes de validación? Relacione la respuesta con la proposición 4.2.

Ejercicio 4.3 ⋆⋆ Una línea de tipo «objeto» se genera eligiendo de forma uniforme uno de 6 objetos, 6 colores, 6 materiales y 6 usos, y mide 58 caracteres incluido el salto de línea. Contando $\ln 4$ nats por la elección del tipo de línea, ¿qué entropía mínima por carácter tiene? Compárela con la del corpus completo y con la pérdida final del modelo.

Ejercicio 4.4 ⋆⋆ Con los datos del cuadro 4.1 al final del entrenamiento: (a) calcule la perplejidad de entrenamiento y la de validación y el cociente entre ambas; (b) con el teorema 4.6 y la entropía del ejemplo 4.7, estime la divergencia media D_{KL} entre la distribución

real y la del modelo en validación; (c) interprete la perplejidad correspondiente a la entropía del corpus.

Ejercicio 4.5 ⋆⋆ Suponga que todos los tensores están en precisión simple. (a) Calcule la memoria que ocupan los parámetros, los gradientes y el estado de AdamW de la variante del mini-GPT con $d = 128$ y $L = 3$, que tiene 609 838 parámetros, sin contar las activaciones. (b) Repita el cálculo para un modelo de $7 \cdot 10^9$ parámetros. ¿Cuántas GPU de 80 GB harían falta, como mínimo, solo para ese estado?

Fuente: adaptado de Stanford CS336 [11], tarea 1, §4.3, problema *adamw_accounting*, apartado (a) (se omiten las activaciones y se usan los tamaños del módulo).

Ejercicio 4.6 ⋆⋆⋆ Sea (C, X) un par aleatorio con distribución conjunta $p(c, x)$ sobre contextos y tokens, y sea $q(x \mid c)$ un modelo con $q(x \mid c) > 0$ siempre que $p(c, x) > 0$. Demuestre que

$$\mathbb{E}_{p}\big[-\ln q(X \mid C)\big] \;\geq\; H(X \mid C) = -\sum_{c,x} p(c, x) \ln p(x \mid c),$$

con igualdad si y solo si $q(\cdot \mid c) = p(\cdot \mid c)$ para todo c con $p(c) > 0$. Explique por qué este es el resultado que acota la pérdida esperada de un modelo de lenguaje.

Tema 5

Generación e hiperparámetros

Generar texto es aplicar la regla de la cadena del módulo 1 con el modelo como calculadora de probabilidades condicionadas: se predice la distribución del token siguiente, se muestrea, se añade el token al contexto y se repite. Este tema formaliza el bucle de generación, estudia las tres transformaciones de la distribución que exponen las interfaces de los modelos actuales (temperatura, top-k y top-p) y demuestra que la entropía de la distribución crece con la temperatura. Termina con el efecto de los hiperparámetros sobre el entrenamiento del mini-GPT. Se sigue el §6 de la tarea 1 de CS336 [11], el §9.5.5 de [16], los artículos de Fan, Lewis y Dauphin [2] y de Holtzman *et al.* [5], y el §11.4 de [4].

5.1 Generación autorregresiva

Definición 5.1 (Bucle de generación). Sea un modelo con contexto máximo T, una secuencia inicial (*prompt*) $w_{1:m}$, un número máximo de tokens nuevos G y una transformación Φ que convierte unos logits en una distribución. Para $g = 1, \dots, G$:

1. recortar el contexto a sus últimos T tokens, $c = w_{\max(1,\, m+g-T):\, m+g-1}$;
2. evaluar el modelo sobre c y tomar los logits z de la última posición;
3. muestrear $w_{m+g} \sim \Phi(z)$ y añadirlo a la secuencia;
4. detenerse si w_{m+g} es el token delimitador.

Por el corolario 2.6, los logits de la última posición son los de la distribución $P_\theta(\cdot \mid c)$. Con $\Phi = \mathrm{softmax}$ se muestrea del propio modelo [11, §6, ecuaciones (21)–(22)]; si en cada paso se elige el token más probable, el procedimiento es la *decodificación voraz* [16, §9.5.5]. Cada paso exige una pasada completa del modelo sobre el contexto; el módulo 19 estudia cómo evitar recalcular lo ya calculado con la caché de claves y valores.

5.2 Temperatura

Definición 5.2 (Softmax con temperatura). Para $\tau > 0$, $\mathrm{softmax}(z, \tau)_i = e^{z_i/\tau} \big/ \sum_j e^{z_j/\tau}$. En este módulo la temperatura se escribe τ para no confundirla con la longitud de contexto T.

Proposición 5.3 (Límites). *Sea $z \in \mathbb{R}^K$ con un único máximo en el índice $i^\star$. Cuando $\tau \to 0^+$, $\mathrm{softmax}(z, \tau) \to e_{i^\star}$; cuando $\tau \to \infty$, $\mathrm{softmax}(z, \tau) \to (1/K, \dots, 1/K)$.*

Demostración. Por la proposición 1.12, $\mathrm{softmax}(z, \tau)_i = e^{(z_i - z_{i^\star})/\tau} \big/ \sum_j e^{(z_j - z_{i^\star})/\tau}$. Si $\tau \to 0^+$, cada exponente con $j \neq i^\star$ tiende a $-\infty$ y el de $i^\star$ es nulo, luego el denominador tiende a 1 y

solo sobrevive el numerador de $i^\star$. Si $\tau \to \infty$, todos los exponentes tienden a 0, los numeradores a 1 y el denominador a K. ∎

Teorema 5.4 (La entropía crece con la temperatura). *Sea $z \in \mathbb{R}^K$ no constante y, para $b > 0$, $p^{(b)} = \mathrm{softmax}(bz)$, de modo que $b = 1/\tau$. Entonces*

$$\frac{d}{db} H\big(p^{(b)}\big) = -b \,\mathrm{Var}_{p^{(b)}}(z) < 0,$$

donde $\mathrm{Var}_p(z) = \sum_i p_i z_i^2 - \big(\sum_i p_i z_i\big)^2$. En consecuencia, $H\big(\mathrm{softmax}(z,\tau)\big)$ es estrictamente creciente en τ.

Demostración. Sea $Z(b) = \sum_j e^{bz_j}$. Como $\ln p_i^{(b)} = bz_i - \ln Z(b)$,

$$H\big(p^{(b)}\big) = -\sum_i p_i^{(b)}\big(bz_i - \ln Z(b)\big) = -b\,\mu(b) + \ln Z(b), \qquad \mu(b) = \sum_i p_i^{(b)} z_i.$$

Se tiene $\frac{d}{db}\ln Z(b) = \sum_j z_j e^{bz_j}/Z(b) = \mu(b)$. Además, $\frac{d}{db}p_i^{(b)} = p_i^{(b)}\big(z_i - \mu(b)\big)$, luego $\mu'(b) = \sum_i p_i^{(b)} z_i\big(z_i - \mu(b)\big) = \mathrm{Var}_{p^{(b)}}(z)$. Derivando, $\frac{d}{db}H = -\mu(b) - b\,\mu'(b) + \mu(b) = -b\,\mathrm{Var}_{p^{(b)}}(z)$. Como todas las $p_i^{(b)}$ son positivas y z no es constante, la varianza es positiva. Finalmente, $b = 1/\tau$ es estrictamente decreciente en τ. ∎

Ejemplo 5.5. Con $z = (2;\ 1;\ 0{,}5;\ -1)$:

τ	p_1	p_2	p_3	p_4	H (nats)
0,5	0,842	0,114	0,042	0,002	0,538
1	0,609	0,224	0,136	0,030	1,014
2	0,434	0,263	0,205	0,097	1,265

La entropía crece con τ hacia $\ln 4 = 1{,}386$, el valor de la distribución uniforme, como afirman la proposición 5.3 y el teorema 5.4.

La temperatura es la misma que la de la distribución de Boltzmann de la física estadística (módulo 1). Holtzman *et al.* observan que bajar la temperatura mejora la calidad del texto generado a costa de reducir su diversidad [5, §3.3].

5.3 Muestreo top-k y top-p

Definición 5.6 (Top-k). Sea $1 \le k \le K$ y sea $I_k(z)$ el conjunto de los índices de los k mayores logits, con los empates resueltos por el índice menor. El *muestreo top-k* usa la distribución $\mathrm{softmax}\big(\mathrm{top}_k(z)\big)$, donde $\mathrm{top}_k(z)_i = z_i$ si $i \in I_k(z)$ y $-\infty$ en otro caso.

Fan, Lewis y Dauphin introdujeron este procedimiento con $k = 10$ para generar relatos y observaron que producía textos menos repetitivos que la búsqueda en haz [2, §5.4]. Con $k = 1$ el muestreo coincide con la decodificación voraz; con $k = K$ no cambia nada. Con $z = (2;\ 1;\ 0{,}5;\ -1)$, $k = 2$ y $\tau = 1$, la distribución pasa a ser $(0{,}731;\ 0{,}269;\ 0;\ 0)$: desaparece la cola de tokens improbables, que con 46 caracteres puede acumular una masa apreciable.

Definición 5.7 (Muestreo de núcleo o top-p). Sea q una distribución sobre $\mathcal{V}$ y $\rho \in (0, 1]$. Sea $V(\rho)$ el menor conjunto de índices tal que $\sum_{j \in V(\rho)} q_j \geq \rho$. El *muestreo de núcleo* usa la distribución

$$\tilde{q}_i = \begin{cases} q_i / \sum_{j \in V(\rho)} q_j, & i \in V(\rho), \\ 0, & \text{en otro caso.} \end{cases}$$

Es la definición de Holtzman *et al.* [5, §3.1, ecuaciones (2) y (3)], reproducida por CS336 [11, §6, ecuación (24)], donde el umbral se denota p. La distribución q suele ser ya la softmax con temperatura.

Proposición 5.8 (Cálculo del núcleo). *Sean $q_{(1)} \geq q_{(2)} \geq \cdots \geq q_{(K)}$ las probabilidades ordenadas y $m = \min\{j : \sum_{i \leq j} q_{(i)} \geq \rho\}$. Todo conjunto de masa al menos ρ tiene al menos m elementos, y el conjunto S_m de los m índices más probables alcanza esa masa. Por tanto, los conjuntos mínimos tienen exactamente m elementos y S_m es, entre ellos, uno de masa máxima.*

Demostración. Un conjunto de j índices tiene masa a lo sumo $\sum_{i \leq j} q_{(i)}$, la suma de las j mayores probabilidades. Si $j < m$, esa suma es menor que ρ por la definición de m, luego ningún conjunto de menos de m elementos alcanza la masa ρ. El conjunto S_m la alcanza, también por la definición de m, y su masa es la máxima posible entre los conjuntos de m elementos por la primera observación con $j = m$. ■

Observación 5.9. El conjunto mínimo no es único en general, aunque no haya empates: con $q = (0{,}5;\ 0{,}3;\ 0{,}2)$ y $\rho = 0{,}6$, tanto $\{1, 2\}$ como $\{1, 3\}$ tienen dos elementos y masa al menos 0,6. Por convenio, $V(\rho)$ designa el conjunto S_m, que es el que se obtiene al ordenar la distribución y acumular probabilidades hasta alcanzar ρ, como prescribe CS336 [11, §6].

La ventaja de top-p sobre top-k es que el número de candidatos se adapta a la forma de la distribución. Con una distribución plana, un k pequeño elimina continuaciones razonables; con una distribución muy concentrada, un k grande deja entrar candidatos inapropiados, cuya probabilidad crece al renormalizar [5, §3.2]. Las interfaces de programación de modelos comerciales como Claude y Gemini exponen públicamente los parámetros de temperatura, top-k y top-p.

Muestras del mini-GPT. Con $\tau = 0{,}8$ y $k = 10$, casi todas las líneas generadas son correctas:

```
objeto. una lámpara azul de cristal. sirve para esperar sentado.
objeto. una caja azul de tela. sirve para leer de noche.
receta. hornea verde paperado.
```

La tercera muestra el límite del modelo: mezcla un color propio de los objetos en una receta. Con $\tau = 3$ la salida se degrada a símbolos sueltos, porque la distribución se acerca a la uniforme y los dígitos y signos aparecen en cualquier lugar.

5.4 Efecto de los hiperparámetros

El cuadro 5.1 resume entrenamientos de 800 pasos sobre el mismo corpus y con la misma semilla, en los que solo cambia lo indicado.

Cuadro 5.1: Efecto de los hiperparámetros en el mini-GPT (resultados del cuaderno de prácticas). Base: $d = 64$, $L = 2$, $n_h = 4$, $T = 32$, $\eta_{\max} = 3 \cdot 10^{-3}$. Pérdidas de validación en nats por carácter; tiempo relativo al modelo base.

Cambio	N	Paso 100	Final	Tiempo	Lectura
Base	107 694	2,046	0,324	1	referencia
$\eta_{\max} = 3 \cdot 10^{-4}$	107 694	2,959	1,840	1	demasiado lenta
$\eta_{\max} = 3 \cdot 10^{-2}$	107 694	1,304	0,297	1	mejor: tolera tasas altas
ídem, sin calentamiento	107 694	1,232	0,306	1	algo peor
$\eta_{\max} = 3 \cdot 10^{-1}$	107 694	2,863	2,244	1	inestable
Tasa constante	107 694	2,046	0,337	1	falta el afinado final
$d = 32$	29 294	2,308	0,649	0,5	poca capacidad
$L = 1$	57 902	2,128	0,392	0,4	un bloque no basta
$n_h = 1$	107 694	2,062	0,336	0,7	algo peor
$T = 8$	106 158	2,068	0,694	0,3	no ve la etiqueta
$d = 128$, $L = 3$	609 838	1,441	0,297	2,4	mejor, más caro

La tasa de aprendizaje es el factor dominante: entre $3 \cdot 10^{-4}$ y $3 \cdot 10^{-2}$ la pérdida final pasa de 1,840 a 0,297, y con $3 \cdot 10^{-1}$ el entrenamiento se rompe. Goodfellow, Bengio y Courville la señalan como el hiperparámetro más importante si solo se puede ajustar uno [4, §11.4.1], y CS336 pide localizar empíricamente la tasa a partir de la cual el entrenamiento diverge y relacionarla con la mejor [11, §7.2, problema *learning_rate*]. El contexto es el segundo factor: con $T = 8$ la pérdida se duplica, porque el modelo pierde la información que más reduce la incertidumbre, el tipo de línea (ejercicio 5.6). Multiplicar por 5,7 los parámetros da la misma mejora que subir la tasa a $3 \cdot 10^{-2}$ sin coste adicional. El calentamiento y el coseno aportan poco en este modelo (unas centésimas de nat cada uno), porque es pequeño y el recorte ya lo protege; en los modelos grandes son imprescindibles.

Observación 5.10 (Reglas prácticas). De lo anterior se desprenden cuatro reglas: (i) ajustar primero la tasa de aprendizaje, barriendo potencias de 10 o de $\sqrt{10}$, y quedarse con la mayor que sea estable; (ii) antes de escalar, comprobar que el modelo sobreajusta un único lote hasta pérdida casi nula; (iii) elegir T de modo que cubra la dependencia que se quiere capturar, aquí una línea completa de unos 60 a 80 caracteres; (iv) más parámetros solo ayudan si hay datos y pasos suficientes para entrenarlos, como precisan las leyes de escalado del módulo 16.

Lecturas recomendadas para este tema

- Stanford CS336 [11], tarea 1, §6 «Generating text» (ecuaciones 21–24) y §7.2 «TinyStories».
- Holtzman *et al.* [5], §3 «Language Model Decoding» (§3.1–3.3).
- Fan, Lewis y Dauphin [2], §5.4 «Generation».
- Zhang *et al.* [16], §9.5.5 «Decoding».
- Goodfellow, Bengio y Courville [4], §11.4 «Selecting Hyperparameters», en particular §11.4.1.
- Karpathy [6], archivos `sample.py` y `config/train_shakespeare_char.py`.

Ejercicios del tema 5

Ejercicio 5.1 ⋆ Describa la distribución de muestreo en cada caso: (a) $k = 1$; (b) $k = K$; (c) $\rho = 1$; (d) $\tau \to 0^+$; (e) $\tau = 3$ con 46 caracteres. ¿Cuáles de ellos son deterministas?

Ejercicio 5.2 ⋆⋆ Con logits $z = (3, 1, 1, 0)$, aplique top-k con $k = 2$ y $\tau = 1$. El segundo y el tercer logit están empatados: obtenga la distribución con el convenio de la definición 5.6 y con el convenio alternativo que conserva todo logit mayor o igual que el k-ésimo mayor.

Ejercicio 5.3 ⋆⋆ Con $z = (3, 1, 0)$, calcule $\mathrm{softmax}(z, \tau)$ y su entropía para $\tau = 0{,}5$, 1 y 2, y compruebe que la entropía crece con τ. Calcule $\mathrm{Var}_p(z)$ para $\tau = 1$ y la derivada dH/db en $b = 1$ del teorema 5.4.

Ejercicio 5.4 ⋆⋆ La distribución del token siguiente es $q = (0{,}5;\ 0{,}2;\ 0{,}15;\ 0{,}1;\ 0{,}05)$. (a) Aplique top-$p$ con $\rho = 0{,}8$ y con $\rho = 0{,}9$. (b) Aplique antes la temperatura $\tau = 0{,}5$ a los logits $\ln q_i$ y después top-p con $\rho = 0{,}8$.

Fuente: adaptado de Stanford CS336 [11], tarea 1, §6, problema *decoding* y ecuaciones (23)–(24) (datos numéricos añadidos).

Ejercicio 5.5 ⋆⋆⋆ (a) Demuestre que, para todo $\tau > 0$, aplicar top-k y después la temperatura da la misma distribución que aplicar la temperatura y después top-k. (b) Demuestre con un contraejemplo que la afirmación análoga para top-p es falsa. (c) ¿Qué orden usa la definición 5.7 cuando q es la softmax con temperatura?

Ejercicio 5.6 ⋆⋆⋆ Con $T = 8$, el mini-GPT termina con pérdida de validación 0,694 frente a 0,324 con $T = 32$ (cuadro 5.1). Use la estructura del corpus del ejemplo 1.2 para explicar qué información le falta, qué partes de la línea siguen siendo predecibles y cuáles se vuelven inciertas. Cuantifique el exceso de pérdida por línea de 58 caracteres y compárelo con la entropía de la etiqueta.

Apéndice A

Soluciones de los ejercicios

Cada solución se presenta en cinco partes: datos, planteamiento, desarrollo, resultado y comprobación. En los pasos intermedios se redondea a cuatro cifras significativas salvo que se indique otra cosa; los resultados exactos se dan como fracción cuando existe.

A.1 Tema 1

Solución del ejercicio 1.1

Datos. ventana $x = (x_1, \ldots, x_T)$ y objetivo y; lote de forma (B, T).

Planteamiento. en la posición t el modelo ve $x_{1:t}$ y debe predecir el token que lo sigue en el corpus (definición 1.3).

Desarrollo.

1. El token que sigue a $x_{1:t}$ en el corpus es x_{t+1} (para $t < T$) o el siguiente al final de la ventana (para $t = T$). Definiendo y_t como ese token, cada posición tiene su respuesta en el mismo índice.
2. Por tanto y se obtiene recortando el corpus un token más tarde: es x desplazada una posición.
3. Cada ventana aporta T predicciones (proposición 1.4) y el lote apila B ventanas: $B \cdot T$ predicciones.
4. Con $B = T = 32$: $32 \cdot 32 = 1024$.

Resultado. $y_t = x_{t+1}$**; un lote** (B, T) **contiene** $B \cdot T$ **predicciones,** 1024 **con** $B = T = 32$**.**

Comprobación. en el ejemplo 1.5, las 6 ventanas con $T = 4$ dan $6 \cdot 4 = 24$ predicciones.

Solución del ejercicio 1.2

Datos. texto «hola`\n`»: $n = 5$ caracteres (`h`, `o`, `l`, `a`, `\n`); $T = 2$.

Planteamiento. por la proposición 1.4 hay $n - T$ ventanas, con inicios $i = 1, \ldots, n - T$.

Desarrollo.

1. Número de ventanas: $5 - 2 = 3$.
2. Inicio 1: $x =$ `ho`, $y =$ `ol`.
3. Inicio 2: $x =$ `ol`, $y =$ `la`.
4. Inicio 3: $x =$ `la`, $y =$ `a\n`.
5. Predicciones: $3 \cdot 2 = 6$.

6. Un inicio en el cuarto carácter daría $x =$ `a\n` y exigiría como último objetivo el sexto carácter, que no existe.

Resultado. **3 ventanas:** (`ho`, `ol`)**,** (`ol`, `la`)**,** (`la`, `a\n`)**; 6 predicciones.**

Comprobación. cada ventana usa $T + 1 = 3$ caracteres consecutivos, y en un texto de 5 hay exactamente $5 - 3 + 1 = 3$ bloques de 3.

Solución del ejercicio 1.3

Datos. $n = 10$ tokens $u_1, \ldots, u_{10}$; $T = 4$; la ventana que empieza en i usa $u_i, \ldots, u_{i+4}$.

Planteamiento. una ventana necesita $T + 1 = 5$ tokens consecutivos; en el corte sin solape, los trozos empiezan en $k + 1$, $k + 1 + T$, …

Desarrollo.

1. (a) Inicios válidos: $i = 1, \ldots, n - T = 6$; hay 6 ventanas, que se denotan $w_1, \ldots, w_6$ por su inicio.
2. (b) Tras descartar k tokens quedan $10 - k$; un trozo usa 4 de entrada y necesita uno más, luego caben $\lfloor (10 - k - 1)/4 \rfloor$ trozos.
3. $k = 0$: $\lfloor 9/4 \rfloor = 2$ trozos (w_1, w_5). $k = 1$: $\lfloor 8/4 \rfloor = 2$ (w_2, w_6). $k = 2$: $\lfloor 7/4 \rfloor = 1$ (w_3). $k = 3$: $\lfloor 6/4 \rfloor = 1$ (w_4).
4. Probabilidades: $P(w_1) = P(w_5) = \frac{1}{4} \cdot \frac{1}{2} = \frac{1}{8}$; $P(w_2) = P(w_6) = \frac{1}{8}$; $P(w_3) = P(w_4) = \frac{1}{4} \cdot 1 = \frac{1}{4}$.
5. No es uniforme: w_3 y w_4 salen con el doble de probabilidad que las demás. Sí lo es el procedimiento de la definición 1.6: elegir el inicio uniformemente entre los 6 posibles, con probabilidad $\frac{1}{6}$ cada uno.
6. (c) Las frases tienen longitudes distintas: para formar un lote rectangular habría que rellenar las cortas (cómputo desperdiciado) o descartar o cortar las largas. Con el corpus concatenado y un delimitador entre documentos, todas las ventanas tienen longitud T y no hace falta relleno [11, §5.1].

Resultado. **(a) 6 ventanas; (b) 2, 2, 1 y 1 trozos para** $k = 0, 1, 2, 3$**, con probabilidades** $\frac{1}{8}, \frac{1}{8}, \frac{1}{4}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$ **para** $w_1, \ldots, w_6$**: no es uniforme, y sí lo es elegir el inicio al azar entre los 6; (c) las frases completas obligan a rellenar, y la concatenación lo evita.**

Comprobación. las probabilidades suman $4 \cdot \frac{1}{8} + 2 \cdot \frac{1}{4} = 1$. En el ejemplo 1.5, también con $n = 10$ y $T = 4$, hay 6 ventanas.

Solución del ejercicio 1.4

Datos. $o = (1000, 1001, 999)$, objetivo el primer componente; pérdidas (0,5; 1,2; 0,1; 2,0).

Planteamiento. proposición 1.12 con $z^\star = 1001$; perplejidad por la definición 1.9.

Desarrollo.

1. $o - z^\star = (-1, 0, -2)$.
2. $e^{-1} = 0{,}3679$, $e^0 = 1$, $e^{-2} = 0{,}1353$; suma 1,503; $\ln 1{,}503 = 0{,}4076$.
3. $\ell = -(-1) + 0{,}4076 = 1{,}408$ nats.
4. En precisión simple el máximo representable es $\approx 3{,}4 \cdot 10^{38} \approx e^{88{,}7}$; e^{1000} da infinito, el cociente ∞/∞ es indefinido (NaN) y la pérdida no puede calcularse.

5. (b) Media: $(0{,}5 + 1{,}2 + 0{,}1 + 2{,}0)/4 = 3{,}8/4 = 0{,}95$; PPL $= e^{0{,}95} = 2{,}586$.

Resultado. **(a)** $\ell = 1{,}408$ **nats, y el cálculo directo desborda; (b)** PPL $\approx 2{,}59$**.**

Comprobación. o es una permutación de $(2, 1, 0) + 999$ y el objetivo es el logit intermedio; con los logits del ejemplo 1.11, $\ell = -1 + \ln 11{,}107 = -1 + 2{,}408 = 1{,}408$.

Solución del ejercicio 1.5

Datos. $\mathcal{L} = 0{,}7$ nats por carácter.

Planteamiento. PPL $= e^{\mathcal{L}}$; bits $=$ nats $/ \ln 2$.

Desarrollo.

1. PPL $= e^{0{,}7} = 2{,}014$.
2. $\ln 2 = 0{,}6931$; $0{,}7/0{,}6931 = 1{,}010$ bits por carácter.
3. En base 2: $2^{1{,}010} = 2{,}014$.

Resultado. PPL $\approx 2{,}01$**;** $1{,}01$ **bits por carácter.**

Comprobación. $2^{\mathcal{L}/\ln 2} = e^{(\mathcal{L}/\ln 2)\ln 2} = e^{\mathcal{L}}$, luego la perplejidad no depende de la base.

Solución del ejercicio 1.6

Datos. capa de salida $z = Wh + b$ con $W = 0$, $b = 0$; K tokens; lote con BT objetivos y frecuencias empíricas $\hat{\pi}$.

Planteamiento. calcular los logits, aplicar la proposición 1.13 y, para el gradiente, la proposición 1.10 con $\partial z/\partial b = I$.

Desarrollo.

1. Para cualquier representación h, $z = 0 \cdot h + 0 = \mathbf{0}$: los logits no dependen de los demás parámetros.
2. Por la proposición 1.13, cada posición tiene $p = (1/K, \ldots, 1/K)$ y pérdida $\ln K$.
3. La pérdida del lote es la media de BT valores iguales a $\ln K$, es decir, $\ln K$.
4. Gradiente respecto de b en una posición: como $z = Wh + b$, $\partial\ell/\partial b = \nabla_z \ell = p - e_y = \frac{1}{K}\mathbf{1} - e_y$.
5. Promediando sobre el lote: $\nabla_b \mathcal{L} = \frac{1}{K}\mathbf{1} - \frac{1}{BT}\sum_{b,t} e_{Y_{b,t}} = \frac{1}{K}\mathbf{1} - \hat{\pi}$.
6. Tras un paso de descenso con tasa η: $b = \eta\,(\hat{\pi} - \frac{1}{K}\mathbf{1})$. Los tokens más frecuentes que la media reciben un logit positivo y los menos frecuentes uno negativo.

Resultado. $\mathcal{L}_0 = \ln K$ **exactamente (**$3{,}829$ **nats con** $K = 46$**), y** $\nabla_b \mathcal{L} = \frac{1}{K}\mathbf{1} - \hat{\pi}$**: el primer paso orienta el sesgo hacia las frecuencias de los tokens, es decir, hacia un modelo de unigramas.**

Comprobación. las componentes de $\frac{1}{K}\mathbf{1} - \hat{\pi}$ suman $1 - 1 = 0$, como las de todo gradiente respecto de los logits (ejemplo 1.11). El mini-GPT de la práctica empieza en $4{,}046$, por encima de $\ln 46$, porque su cabeza no se inicializa a cero.

A.2 Tema 2

Solución del ejercicio 2.1

Datos. modelo de dos o más bloques; máscara $\boldsymbol{M}$ de la definición 2.1.

Planteamiento. separar el entrenamiento sin máscara (fuga de la respuesta) de la generación sin máscara (desajuste entre entrenamiento y uso).

Desarrollo.

1. Sin máscara al entrenar, la posición t puede atender a $x_{t+1} = y_t$. Una cabeza que copie la posición siguiente reduce la pérdida casi a cero sin aprender a predecir (observación 2.7).
2. Ese modelo falla al generar: en la última posición no existe el token siguiente, y la pista de la que dependía desaparece.
3. Con máscara al entrenar y sin ella al generar: en el primer bloque, la última posición no cambia, porque no hay posiciones posteriores a las que atender.
4. Sí cambian las representaciones de las posiciones anteriores, que ahora atienden también a posiciones posteriores.
5. Desde el segundo bloque, la última posición lee esas representaciones distintas, que nunca vio durante el entrenamiento, y su predicción se degrada.

Resultado. **sin máscara al entrenar, el modelo copia la respuesta (pérdida casi nula que no generaliza); sin máscara solo al generar, el primer bloque de la última posición no cambia, pero los siguientes reciben entradas distintas de las del entrenamiento.**

Comprobación. el cuadro 4.2 asocia una pérdida casi nula en entrenamiento y validación con la fuga del futuro.

Solución del ejercicio 2.2

Datos. puntuaciones S por filas $(0, 1, 3)$, $(1, 1, 2)$, $(2, 0, 1)$; valores $v_1 = (1, 0)$, $v_2 = (0, 1)$, $v_3 = (1, 1)$.

Planteamiento. por el lema 2.3, la fila t es la softmax de $(S_{t1}, \ldots, S_{tt})$; la salida en t es $\sum_{j \le t} a_{tj} v_j$.

Desarrollo.

1. Fila 1: solo $j = 1$; $a_{1\cdot} = (1, 0, 0)$.
2. Fila 2: softmax$(1, 1) = (0{,}5;\ 0{,}5)$; $a_{2\cdot} = (0{,}5;\ 0{,}5;\ 0)$.
3. Fila 3: $e^2 = 7{,}389$, $e^0 = 1$, $e^1 = 2{,}718$; suma $11{,}107$; $a_{3\cdot} = (0{,}6652;\ 0{,}0900;\ 0{,}2447)$.
4. Salidas: $o_1 = v_1 = (1, 0)$; $o_2 = 0{,}5\,(1, 0) + 0{,}5\,(0, 1) = (0{,}5;\ 0{,}5)$.
5. $o_3 = 0{,}6652\,(1, 0) + 0{,}0900\,(0, 1) + 0{,}2447\,(1, 1) = (0{,}6652 + 0{,}2447;\ 0{,}0900 + 0{,}2447) = (0{,}9100;\ 0{,}3348)$.
6. No influyen las entradas por encima de la diagonal: $S_{12} = 1$, $S_{13} = 3$ y $S_{23} = 2$.

Resultado. **pesos** $\begin{pmatrix} 1 & 0 & 0 \\ 0{,}5 & 0{,}5 & 0 \\ 0{,}6652 & 0{,}0900 & 0{,}2447 \end{pmatrix}$**; salidas** $(1, 0)$**,** $(0{,}5;\ 0{,}5)$ **y** $(0{,}9100;\ 0{,}3348)$**; las puntuaciones** S_{12}**,** S_{13} **y** S_{23} **se ignoran.**

Comprobación. cada fila suma 1 ($0{,}6652 + 0{,}0900 + 0{,}2447 = 0{,}9999$ por redondeo). La mayor puntuación de toda la matriz, $S_{13} = 3$, no tiene ningún efecto, como exige la causalidad.

Solución del ejercicio 2.3

Datos. $K = 46$, $T = 32$, $d = 32$, $L = 2$.

Planteamiento. proposición 2.8: $N = Kd + Td + L(12d^2 + 10d) + 2d + dK + K$; con atadura se resta Kd (observación 2.9).

Desarrollo.

1. Bloque: $12 \cdot 32^2 = 12 \cdot 1024 = 12\,288$; $10 \cdot 32 = 320$; total $12\,608$.
2. Dos bloques: $2 \cdot 12\,608 = 25\,216$.
3. Embeddings: $46 \cdot 32 = 1472$ (token) y $32 \cdot 32 = 1024$ (posición).
4. Normalización final: $2 \cdot 32 = 64$. Cabeza: $32 \cdot 46 + 46 = 1518$.
5. Total: $1472 + 1024 + 25\,216 + 64 + 1518 = 29\,294$.
6. Bloques: $25\,216/99\,584 = 0{,}2532$ de los del modelo base.
7. Con pesos atados: $29\,294 - 1472 = 27\,822$.

Resultado. $N = 29\,294$ **(o 27 822 con pesos atados); los bloques tienen aproximadamente la cuarta parte de parámetros que con** $d = 64$**.**

Comprobación. el cociente 0,2532 es cercano a $(32/64)^2 = 0{,}25$, porque el término dominante es $12d^2$; coincide con la fila $d = 32$ del cuadro 5.1.

Solución del ejercicio 2.4

Datos. modelo base con $d = 64$ y $n_h = 4$ frente a $n_h = 1$; pérdidas finales 0,324 y 0,336.

Planteamiento. la proposición 2.8 no depende de n_h; $d_k = d/n_h$.

Desarrollo.

1. (a) Ambos tienen $N = 107\,694$ parámetros.
2. (b) Con $n_h = 4$, $d_k = 64/4 = 16$ y cada bloque calcula 4 matrices de pesos $T \times T$. Con $n_h = 1$, $d_k = 64$ y una sola matriz.
3. (c) Con el mismo número de parámetros, la diferencia está en la estructura: cada cabeza produce en cada posición *una* combinación convexa de valores. Con una cabeza, la posición t solo puede promediar; con cuatro, puede atender a la vez, por ejemplo, a la etiqueta de la línea y al carácter anterior, y combinar ambos resultados mediante W_O.

Resultado. **(a) 107 694 en ambos casos; (b) 4 matrices de pesos con** $d_k = 16$ **frente a una con** $d_k = 64$**; (c) la mejora procede de poder atender a varias posiciones a la vez, no de tener más parámetros.**

Comprobación. con los mismos parámetros, la variante de una cabeza es más rápida en la práctica (tiempo relativo 0,7): el número de parámetros no mide el coste de calcular y aplicar varias matrices de atención.

Solución del ejercicio 2.5

Datos. $K = 50\,257$, $T = 1024$, $d = 768$, $L = 12$; sesgos en Q, K, V; cabeza atada a E y sin sesgo.

Planteamiento. adaptar la proposición 2.8: el sesgo de Q, K, V añade $3d$ por bloque, y la cabeza no añade nada.

Desarrollo.

1. Bloque: $3d^2 + 3d$ (QKV) $+ d^2 + d$ (salida) $+ 8d^2 + 5d$ (MLP) $+ 4d$ (normalizaciones) $= 12d^2 + 13d$.
2. $d^2 = 589\,824$; $12d^2 = 7\,077\,888$; $13d = 9984$; bloque $= 7\,087\,872$.
3. Doce bloques: $12 \cdot 7\,087\,872 = 85\,054\,464$.
4. Embeddings: $Kd = 50\,257 \cdot 768 = 38\,597\,376$; $Td = 1024 \cdot 768 = 786\,432$.
5. Normalización final: $2d = 1536$. Total: $85\,054\,464 + 38\,597\,376 + 786\,432 + 1536 = 124\,439\,808$.
6. (b) Memoria: $124\,439\,808 \cdot 4 = 497\,759\,232$ bytes ≈ 498 MB.
7. (c) Fracciones:

$$\frac{85\,054\,464}{124\,439\,808} = 0{,}683 \text{ (bloques)}, \qquad \frac{38\,597\,376 + 786\,432}{124\,439\,808} = 0{,}316 \text{ (embeddings)}.$$

Resultado. $N = 124\,439\,808$ **(≈ 124 millones), ≈ 498 MB en precisión simple; el 68 % de los parámetros está en los bloques y el 32 % en las tablas de embeddings, frente al 92 % en bloques del mini-GPT.**

Comprobación. el resultado coincide con el tamaño de 124 millones con que se suele citar GPT-2 *small* (la tabla 2 de [10] lo redondea a 117 millones). La fracción de embeddings es alta porque $K = 50\,257$ es grande frente a $d = 768$; con pesos sin atar habría $38\,597\,376$ parámetros más.

Solución del ejercicio 2.6

Datos. mini-GPT con $P = 0$; permutación σ; $x^\sigma_t = x_{\sigma(t)}$.

Planteamiento. inducción sobre las capas, como en el teorema 2.5, usando que una suma no depende del orden de sus términos.

Desarrollo.

1. (a) Base: $h^0_t(x^\sigma) = E_{x_{\sigma(t)}} = h^0_{\sigma(t)}(x)$; las filas de $H^0(x^\sigma)$ son las de $H^0(x)$ permutadas.
2. Paso: supóngase $h^{\ell-1}_t(x^\sigma) = h^{\ell-1}_{\sigma(t)}(x)$ para todo t. Las operaciones fila a fila (normalización, proyecciones, MLP, residuales) conservan esta relación.
3. Sin máscara, la cabeza r da en la fila t la suma $\sum_{j=1}^{T} a_{tj}(\boldsymbol{V}_r)_j$, con $a_{tj} \propto \exp\big((\boldsymbol{Q}_r)_t \cdot (\boldsymbol{K}_r)_j/\sqrt{d_k}\big)$. Para x^σ, la consulta es la de la posición $\sigma(t)$ de x, y la suma recorre las claves y valores de todas las posiciones de x en otro orden; su valor coincide con el de la fila $\sigma(t)$ para x. Por inducción, $z_t(x^\sigma) = z_{\sigma(t)}(x)$.
4. (b) Con máscara y $L = 1$, la posición T atiende a todas las posiciones $j \le T$, es decir, a todas. Si $\sigma(T) = T$, su consulta no cambia ($h^0_T = E_{x_T}$), y el conjunto de pares (clave, valor) es el mismo, reordenado. La suma, y con ella z_T, no cambia.
5. (c) Con $L \ge 2$, la segunda capa en la posición T lee las representaciones h^1_j de la primera, y cada h^1_j depende del prefijo $x_{1:j}$ por la máscara. Al permutar, los prefijos cambian como conjuntos (por ejemplo, h^1_1 pasa a depender de $x_{\sigma(1)}$ en lugar de x_1), de modo que el conjunto $\{h^1_j\}$ no es una reordenación del original.

6. La máscara rompe, por tanto, la simetría de permutación: la posición j atiende a exactamente j tokens, y esa asimetría permite al modelo inferir el orden sin codificación posicional. CS336 propone comprobarlo empíricamente con el experimento *no_pos_emb* (NoPE) de su §7.3 [11].

Resultado. **(a) sin máscara y sin posiciones, el modelo es equivariante por permutaciones para todo L; (b) con máscara y un bloque, z_T es invariante frente a las permutaciones que fijan T; (c) con dos o más bloques la máscara introduce información de orden, por lo que un decoder puede entrenarse sin codificación posicional.**

Comprobación. para $T = 2$ en (a): intercambiar x_1 y x_2 intercambia z_1 y z_2, y en (b) con $T = 3$, intercambiar x_1 y x_2 deja invariante z_3 pero no z_2, que pasa de ver $\{x_1, x_2\}$ con consulta x_2 a verlo con consulta x_1.

A.3 Tema 3

Solución del ejercicio 3.1

Planteamiento. revisar qué hace cada paso de la definición 3.1 y qué cantidades modifica.

Desarrollo.

1. El recorte necesita el gradiente completo, luego va después de la retropropagación. Debe ir antes del paso del optimizador porque AdamW incorpora g a sus medias móviles: un gradiente anómalo sin recortar contaminaría m y v durante muchos pasos (unos $1/(1-\beta_2)$ en el caso de v).
2. Con la norma global, el gradiente recortado es un múltiplo positivo del original y conserva la dirección de descenso (proposición 3.10); por tensor, la dirección cambia.
3. El calentamiento evita pasos grandes cuando los momentos de Adam se han estimado con pocos gradientes y estos son grandes e irregulares.
4. El decaimiento coseno reduce la tasa al final, lo que disminuye el exceso de pérdida debido al ruido del gradiente estocástico y permite afinar.

Resultado. **recorte entre `backward` y `step`, con norma global para no cambiar la dirección; el calentamiento protege el arranque y el coseno afina el final.**

Comprobación. en el cuadro 5.1, quitar el calentamiento empeora la pérdida final de 0,297 a 0,306 y quitar el coseno, de 0,324 a 0,337.

Solución del ejercicio 3.2

Datos. $W = 200$, $S = 2000$, $\eta_{\max} = 10^{-3}$, $\eta_{\min} = 10^{-4}$; $s = 9$ y $s = 1000$.

Planteamiento. definición 3.5: rama lineal si $s < W$; rama coseno si no.

Desarrollo.

1. $s = 9 < 200$: $\eta_9 = 10^{-3} \cdot 10/200 = 5 \cdot 10^{-5}$.
2. $s = 1000 \geq 200$: progreso $(1000 - 200)/(2000 - 200) = 800/1800 = 0{,}4444$.
3. Ángulo $\pi \cdot 0{,}4444 = 1{,}396$ rad; $\cos 1{,}396 = 0{,}1736$.

4. $\frac{1}{2}(\eta_{\max} - \eta_{\min}) = \frac{1}{2} \cdot 9 \cdot 10^{-4} = 4{,}5 \cdot 10^{-4}$; $1 + 0{,}1736 = 1{,}1736$.
5. $\eta_{1000} = 10^{-4} + 4{,}5 \cdot 10^{-4} \cdot 1{,}1736 = 10^{-4} + 5{,}281 \cdot 10^{-4} = 6{,}281 \cdot 10^{-4}$.

Resultado. $\eta_9 = 5 \cdot 10^{-5}$ **y** $\eta_{1000} \approx 6{,}28 \cdot 10^{-4}$**.**

Comprobación. el punto medio del coseno es $s = 1100$, con $\eta = 5{,}5 \cdot 10^{-4}$; $s = 1000$ está antes y su tasa es mayor. Por la proposición 3.6 (iv), $\eta_{1200} = 1{,}1 \cdot 10^{-3} - 6{,}281 \cdot 10^{-4} = 4{,}719 \cdot 10^{-4}$, que coincide con el cálculo directo.

Solución del ejercicio 3.3

Datos. $\alpha_{\max} = 10^{-3}$, $\alpha_{\min} = 10^{-4}$, $T_w = 100$, $T_c = 1000$; $\alpha_{\max} - \alpha_{\min} = 9 \cdot 10^{-4}$.

Planteamiento. aplicar cada rama de la observación 3.8; en el coseno, la fracción es $(t - 100)/900$.

Desarrollo.

1. $t = 0$: $(0/100) \cdot 10^{-3} = 0$. $t = 50$: $(50/100) \cdot 10^{-3} = 5 \cdot 10^{-4}$.
2. $t = 100$: fracción 0, $\cos 0 = 1$; $\alpha = 10^{-4} + \frac{1}{2} \cdot 2 \cdot 9 \cdot 10^{-4} = 10^{-3}$.
3. $t = 325$: fracción 0,25; $\cos(\pi/4) = 0{,}7071$; $\frac{1}{2} \cdot 1{,}7071 \cdot 9 \cdot 10^{-4} = 7{,}682 \cdot 10^{-4}$; $\alpha = 8{,}682 \cdot 10^{-4}$.
4. $t = 550$: fracción 0,5; $\cos(\pi/2) = 0$; $\alpha = 10^{-4} + 4{,}5 \cdot 10^{-4} = 5{,}5 \cdot 10^{-4}$.
5. $t = 775$: fracción 0,75; $\cos(3\pi/4) = -0{,}7071$; $\frac{1}{2} \cdot 0{,}2929 \cdot 9 \cdot 10^{-4} = 1{,}318 \cdot 10^{-4}$; $\alpha = 2{,}318 \cdot 10^{-4}$.
6. $t = 1000$: $\cos \pi = -1$; $\alpha = 10^{-4}$. $t = 1200 > T_c$: $\alpha = \alpha_{\min} = 10^{-4}$.

Resultado. 0**;** $5 \cdot 10^{-4}$**;** 10^{-3}**;** $8{,}682 \cdot 10^{-4}$**;** $5{,}5 \cdot 10^{-4}$**;** $2{,}318 \cdot 10^{-4}$**;** 10^{-4}**;** 10^{-4}**.** CS336 empieza en $\alpha_0 = 0$ y llega al pico en $t = T_w$, y mantiene $\alpha_{\min}$ después de T_c; el calendario del módulo empieza en $\eta_{\max}/W$, llega al pico en $s = W - 1$ y termina en $s = S - 1$.

Comprobación. $t = 325$ y $t = 775$ son simétricos respecto del punto medio: $8{,}682 \cdot 10^{-4} + 2{,}318 \cdot 10^{-4} = 1{,}1 \cdot 10^{-3} = \alpha_{\max} + \alpha_{\min}$, como en la proposición 3.6 (iv).

Solución del ejercicio 3.4

Datos. $c = 1$, $\varepsilon = 10^{-6}$.

Planteamiento. calcular $\|g\|_2$; si es al menos c, multiplicar por $c/(\|g\|_2 + \varepsilon)$ (definición 3.9).

Desarrollo.

1. (a) $\|(3, 4, 0)\|_2 = \sqrt{9 + 16} = 5 \geq 1$. Factor $1/(5 + 10^{-6}) = 0{,}2000$; $g' = (0{,}6;\ 0{,}8;\ 0)$, de norma $5/(5 + 10^{-6}) = 0{,}9999998$.
2. (b) $\|(0{,}3;\ 0{,}4;\ 0)\|_2 = \sqrt{0{,}09 + 0{,}16} = 0{,}5 < 1$: no se modifica.
3. (c) Norma global: $\sqrt{3^2 + 4^2 + 12^2} = \sqrt{169} = 13$. Factor $1/13 = 0{,}07692$: $(0{,}2308;\ 0{,}3077)$ y $(0{,}9231)$.
4. Fracción de la norma del primer tensor: antes y después del recorte global, $5/13 = 0{,}3846$.
5. Por tensor: $(3, 4) \to (0{,}6;\ 0{,}8)$ con norma 1 y $(12) \to (1)$. Norma total $\sqrt{2} = 1{,}414$; la fracción del primer tensor pasa a $1/\sqrt{2} = 0{,}7071$.

Resultado. **(a)** $(0{,}6;\ 0{,}8;\ 0)$**; (b) sin cambios; (c)** $(0{,}2308;\ 0{,}3077)$ **y** $(0{,}9231)$**. El recorte global mantiene la fracción** $0{,}385$ **del primer tensor; el recorte por tensor la eleva a** $0{,}707$ **y cambia la dirección.**

Comprobación. $0{,}2308^2 + 0{,}3077^2 + 0{,}9231^2 = 0{,}0533 + 0{,}0947 + 0{,}8521 = 1{,}000$.

Solución del ejercicio 3.5

Datos. $L(w) = \frac{1}{100}\sum_i w_i^2$; tasa $\alpha/\sqrt{t+1}$ en el paso t.

Planteamiento. la actualización es lineal en w, y L es cuadrática: si todos los pesos se multiplican por f, L se multiplica por f^2.

Desarrollo.

1. (a) $\partial L/\partial w_i = 2w_i/100 = w_i/50$. Luego $w \leftarrow w - \frac{\alpha}{\sqrt{t+1}} \cdot \frac{w}{50} = \left(1 - \frac{\alpha}{50\sqrt{t+1}}\right) w = f_t\, w$.
2. (b) $L_3/L_0 = (f_0 f_1 f_2)^2$, con $\sqrt{2} = 1{,}414$ y $\sqrt{3} = 1{,}732$.
3. $\alpha = 1$: $f = (0{,}98;\ 0{,}9859;\ 0{,}9885)$; producto $0{,}9551$; cociente $0{,}912$.
4. $\alpha = 10$: $f = (0{,}8;\ 0{,}8586;\ 0{,}8845)$; producto $0{,}6075$; cociente $0{,}369$.
5. $\alpha = 100$: $f = (-1;\ -0{,}4142;\ -0{,}1547)$; producto $-0{,}06408$; cociente $0{,}0041$.
6. $\alpha = 1000$: $f = (-19;\ -13{,}14;\ -10{,}55)$; producto en valor absoluto 2634; cociente $6{,}94 \cdot 10^6$.

Resultado. $L_3/L_0 = 0{,}912$**,** $0{,}369$**,** $0{,}0041$ **y** $6{,}9\cdot 10^6$**: las tres primeras tasas hacen decrecer la pérdida (la tercera, tras un primer paso que no la cambia) y** $\alpha = 1000$ **diverge.**

Comprobación. la pérdida decrece en el paso t si y solo si $|f_t| < 1$, es decir, $0 < \alpha < 100\sqrt{t+1}$. Con $\alpha = 100$, $f_0 = -1$: el peso cambia de signo y L no varía; con $\alpha = 1000$, $|f_t| > 1$ en los tres pasos.

Solución del ejercicio 3.6

Datos. $N = 107\,694$; 2000 pasos; $B = T = 32$; rendimiento efectivo $3 \cdot 10^{10}$ FLOP/s.

Planteamiento. $C \approx 6ND$ con $D = \text{pasos} \cdot B \cdot T$.

Desarrollo.

1. $D = 2000 \cdot 1024 = 2\,048\,000$ tokens.
2. $6N = 646\,164$; $C = 646\,164 \cdot 2\,048\,000 = 1{,}323 \cdot 10^{12}$ FLOP.
3. Tiempo: $1{,}323 \cdot 10^{12}/(3 \cdot 10^{10}) = 44{,}1$ s.

Resultado. $C \approx 1{,}32 \cdot 10^{12}$ **FLOP; unos 44 s.**

Comprobación. son $2000/800 = 2{,}5$ veces los pasos del ejemplo 3.11: $2{,}5 \cdot 5{,}29 \cdot 10^{11} = 1{,}32 \cdot 10^{12}$ y $2{,}5 \cdot 17{,}6 = 44$ s.

Solución del ejercicio 3.7

Datos. $\mathcal{B}_c = \{u : \|u\|_2 \le c\}$; g arbitrario.

Planteamiento. acotar $\|g - u\|_2$ inferiormente con la desigualdad triangular y encontrar el punto que alcanza la cota.

Desarrollo.

1. Si $\|g\|_2 \le c$, entonces $g \in \mathcal{B}_c$ y su distancia a sí mismo es 0: $\Pi_c(g) = g$.
2. Si $\|g\|_2 > c$, para todo $u \in \mathcal{B}_c$: $\|g - u\|_2 \ge \|g\|_2 - \|u\|_2 \ge \|g\|_2 - c$.
3. El punto $u^\star = c\,g/\|g\|_2$ está en $\mathcal{B}_c$ y $\|g - u^\star\|_2 = (1 - c/\|g\|_2)\|g\|_2 = \|g\|_2 - c$: alcanza la cota.
4. Unicidad: la igualdad en la primera desigualdad exige que u sea un múltiplo no negativo de g, y en la segunda que $\|u\|_2 = c$; el único punto que cumple ambas es $u^\star$.
5. Con $\varepsilon = 0$, la definición 3.9 da g si $\|g\|_2 < c$ y $c\,g/\|g\|_2$ si $\|g\|_2 \ge c$ (en $\|g\|_2 = c$ ambas expresiones coinciden): es Π_c.
6. Por tensor: $\|g - u\|_2^2 = \sum_q \|g^{(q)} - u^{(q)}\|_2^2$ es una suma de términos que dependen de bloques distintos, y cada uno se minimiza por separado proyectando $g^{(q)}$ sobre su bola.

Resultado. **el recorte global con $\varepsilon = 0$ es la proyección euclídea sobre la bola $\mathcal{B}_c$; el recorte por tensor es la proyección sobre el producto cartesiano $\mathcal{B}_c^{(1)} \times \cdots \times \mathcal{B}_c^{(P)}$ de una bola por tensor.**

Comprobación. con $g = (3, 4, 12)$ y $c = 1$, $u^\star = (3, 4, 12)/13$ está a distancia $13 - 1 = 12$ de g, mientras que el recorte por tensor (0,6; 0,8; 1) está a distancia $\sqrt{2{,}4^2 + 3{,}2^2 + 11^2} = \sqrt{137} = 11{,}70$, menor porque pertenece al producto de bolas, un conjunto mayor que $\mathcal{B}_1$, pero fuera de $\mathcal{B}_1$, ya que su norma es $\sqrt{2} > 1$.

A.4 Tema 4

Solución del ejercicio 4.1

Datos. AdamW guarda m, v y el contador t; el calendario depende del paso s.

Planteamiento. analizar qué ocurre al reanudar si esos datos se pierden.

Desarrollo.

1. Sin el estado del optimizador, $m = v = 0$ y $t = 0$: el primer paso usa momentos estimados con un único gradiente.
2. Ese paso tiene la magnitud de un paso de Adam normalizado sin la tasa reducida del calentamiento, y su dirección puede ser mala: la pérdida da un salto y tarda en recuperarse.
3. Sin el número de paso, el calendario vuelve a empezar: se repite el calentamiento y la tasa vuelve a subir hasta $\eta_{\max}$ cuando ya debería ser pequeña.

Resultado. **sin el estado del optimizador, el primer paso tras reanudar es anómalo y la curva da un salto; sin el número de paso, el calendario se reinicia.**

Comprobación. el estado ocupa dos números por parámetro, el doble que el modelo, coherente con los $16N$ bytes de la proposición 4.4 ($4N$ del modelo, $4N$ de los gradientes y $8N$ del estado).

Solución del ejercicio 4.2

Datos. pérdida estimada como media de 20 lotes de validación fijos.

Planteamiento. proposición 4.2.

Desarrollo.

1. La pérdida de un lote es una estimación ruidosa de la pérdida esperada.
2. Con lotes distintos en cada evaluación, la diferencia entre dos puntos de la curva mezcla el cambio del modelo con el azar del muestreo, y su varianza es $\mathrm{Var}(\hat{\mathcal{L}}_a) + \mathrm{Var}(\hat{\mathcal{L}}_b)$.
3. Con los mismos lotes, las dos estimaciones están positivamente correlacionadas (los lotes difíciles lo son para ambos estados) y el término $-2\,\mathrm{Cov}$ reduce la varianza de la diferencia.

Resultado. **con lotes fijos, la diferencia entre evaluaciones refleja sobre todo el cambio del modelo, porque el ruido común se cancela.**

Comprobación. en el caso extremo $\theta_a = \theta_b$, con lotes comunes la diferencia es exactamente 0, mientras que con lotes independientes es una variable aleatoria de varianza $2\,\mathrm{Var}(\hat{\mathcal{L}}_a)$.

Solución del ejercicio 4.3

Datos. 4 elecciones uniformes de 6 opciones; 4 tipos de línea; 58 caracteres.

Planteamiento. las elecciones son uniformes e independientes y el resto de caracteres está determinado: la entropía de la línea es el logaritmo del número de posibilidades.

Desarrollo.

1. Posibilidades: $6^4 = 1296$; $\ln 1296 = 4 \ln 6 = 4 \cdot 1{,}792 = 7{,}167$ nats.
2. Tipo de línea: $\ln 4 = 1{,}386$ nats. Total: $7{,}167 + 1{,}386 = 8{,}553$ nats.
3. Por carácter: $8{,}553/58 = 0{,}1475$ nats.

Resultado. $\approx 0{,}147$ **nats por carácter.**

Comprobación. es del mismo orden que la $H \approx 0{,}125$ del corpus completo (ejemplo 4.7) y muy inferior a los 0,324 del modelo.

Solución del ejercicio 4.4

Datos. entrenamiento 0,264; validación 0,324; $H \approx 0{,}125$ nats por carácter.

Planteamiento. $\mathrm{PPL} = e^{\mathcal{L}}$; por el teorema 4.6, en media, $\mathcal{L}_{\mathrm{val}} = H + D_{\mathrm{KL}}$.

Desarrollo.

1. (a) $e^{0{,}264} = 1{,}302$ y $e^{0{,}324} = 1{,}383$; cociente $e^{0{,}324-0{,}264} = e^{0{,}06} = 1{,}062$.
2. (b) $D_{\mathrm{KL}} \approx 0{,}324 - 0{,}125 = 0{,}199$ nats por carácter.
3. (c) $e^{0{,}125} = 1{,}133$.

Resultado. **(a) perplejidades** 1,30 **y** 1,38**, con cociente** 1,06**; (b)** $D_{\mathrm{KL}} \approx 0{,}20$ **nats por carácter; (c) un modelo perfecto dudaría en media entre** 1,13 **opciones por carácter: casi todos los caracteres están determinados y la incertidumbre se concentra en unas pocas elecciones por línea.**

Comprobación. la brecha de 0,06 nats es pequeña frente a la distancia a la cota, 0,199: el margen de mejora está más en la capacidad y el entrenamiento que en la regularización.

Solución del ejercicio 4.5

Datos. $N = 609\,838$ y $N = 7 \cdot 10^9$; 4 bytes por número; GPU de 80 GB.

Planteamiento. proposición 4.4: $16N$ bytes.

Desarrollo.

1. (a) Parámetros: $4 \cdot 609\,838 = 2\,439\,352$ bytes $= 2{,}44$ MB; lo mismo los gradientes.
2. Estado de AdamW: $8{\cdot}609\,838 = 4\,878\,704$ bytes $= 4{,}88$ MB. Total: $16{\cdot}609\,838 = 9\,757\,408$ bytes $\approx 9{,}76$ MB.
3. (b) $16 \cdot 7 \cdot 10^9 = 1{,}12 \cdot 10^{11}$ bytes $= 112$ GB.
4. GPU necesarias: $112/80 = 1{,}4$, luego al menos 2.

Resultado. **(a)** $\approx 9{,}76$ **MB; (b)** 112 **GB, que exigen al menos 2 GPU de** 80 **GB solo para parámetros, gradientes y estado del optimizador.**

Comprobación. $9{,}76$ MB es $609\,838/107\,694 = 5{,}66$ veces el valor del modelo base, $1{,}72$ MB: $5{,}66 \cdot 1{,}72 = 9{,}75$.

Solución del ejercicio 4.6

Datos. distribución conjunta $p(c, x)$; modelo $q(x \mid c)$ positivo donde $p(c, x) > 0$.

Planteamiento. descomponer la esperanza por contextos y aplicar el teorema 4.6 a cada uno.

Desarrollo.

1. $\mathbb{E}_p[-\ln q(X \mid C)] = \sum_c p(c) \sum_x p(x \mid c)\big(-\ln q(x \mid c)\big) = \sum_c p(c)\, H\big(p(\cdot \mid c), q(\cdot \mid c)\big)$.
2. Por el teorema 4.6, para cada c con $p(c) > 0$: $H\big(p(\cdot \mid c), q(\cdot \mid c)\big) = H\big(p(\cdot \mid c)\big) + D_{\mathrm{KL}}\big(p(\cdot \mid c) \| q(\cdot \mid c)\big)$.
3. Sumando: $\mathbb{E}_p[-\ln q(X \mid C)] = H(X \mid C) + \sum_c p(c)\, D_{\mathrm{KL}}\big(p(\cdot \mid c) \| q(\cdot \mid c)\big)$, porque $\sum_c p(c) H(p(\cdot \mid c)) = H(X \mid C)$.
4. Cada divergencia es no negativa y el peso $p(c)$ también, luego la suma es no negativa: se obtiene la desigualdad.
5. Igualdad si y solo si cada término es nulo, es decir, $D_{\mathrm{KL}}\big(p(\cdot \mid c) \| q(\cdot \mid c)\big) = 0$ para todo c con $p(c) > 0$, lo que ocurre si y solo si $q(\cdot \mid c) = p(\cdot \mid c)$.
6. En un modelo de lenguaje, C es el contexto $x_{1:t}$ y X el token siguiente; la pérdida esperada de la definición 1.8 es exactamente $\mathbb{E}_p[-\ln q(X \mid C)]$.

Resultado. $\mathbb{E}_p[-\ln q(X \mid C)] = H(X \mid C) + \mathbb{E}_C\big[D_{\mathrm{KL}}\big(p(\cdot \mid C) \| q(\cdot \mid C)\big)\big] \geq H(X \mid C)$**, con igualdad si y solo si el modelo coincide con la distribución condicionada real en todo contexto posible.**

Comprobación. si C es constante, $H(X \mid C) = H(X)$ y se recupera el teorema 4.6.

A.5 Tema 5

Solución del ejercicio 5.1

Planteamiento. aplicar las definiciones 5.6 y 5.7 y la proposición 5.3.

Desarrollo.

(a) $k = 1$: solo sobrevive el mayor logit; se elige siempre el token más probable (decodificación voraz).

(b) $k = K$: no se elimina ningún logit; es el muestreo del modelo.

(c) $\rho = 1$: $V(1)$ es el menor conjunto de masa 1, el de los tokens con probabilidad positiva, que con la softmax son todos; no cambia nada.

(d) $\tau \to 0^+$: la distribución tiende al vector *one-hot* del mayor logit, como la voraz.

(e) $\tau = 3$: la distribución se acerca a la uniforme y los 46 caracteres, incluidos dígitos y signos, salen con probabilidades parecidas; el texto se degrada.

Resultado. **(a) y (d) son deterministas (salvo empates); (b) y (c) equivalen a muestrear del modelo; (e) aproxima el muestreo uniforme.**

Comprobación. por el teorema 5.4, la entropía con $\tau = 3$ es mayor que con $\tau = 1$, y está acotada por $\ln 46 = 3{,}829$.

Solución del ejercicio 5.2

Datos. $z = (3, 1, 1, 0)$; $k = 2$; $\tau = 1$.

Planteamiento. quedarse con los índices de $I_2(z)$, poner el resto a $-\infty$ y aplicar la softmax.

Desarrollo.

1. Ordenados: $3, 1, 1, 0$; el segundo y el tercero empatan.
2. Convenio de la definición 5.6 (empate al índice menor): $I_2 = \{1, 2\}$. $e^3 = 20{,}09$, $e^1 = 2{,}718$; suma 22,80; $p = (0{,}8808;\ 0{,}1192;\ 0;\ 0)$.
3. Convenio alternativo (todo logit ≥ 1): sobreviven los índices 1, 2 y 3. Suma $20{,}09 + 2 \cdot 2{,}718 = 25{,}52$; $p = (0{,}7870;\ 0{,}1065;\ 0{,}1065;\ 0)$.

Resultado. $(0{,}881;\ 0{,}119;\ 0;\ 0)$ **con desempate por índice y** $(0{,}787;\ 0{,}107;\ 0{,}107;\ 0)$ **conservando los empatados.**

Comprobación. ambas suman 1, y la primera coincide con la que devuelve una implementación que elige exactamente k índices. Con logits reales, los empates exactos son raros.

Solución del ejercicio 5.3

Datos. $z = (3, 1, 0)$; $\tau = 0{,}5,\ 1,\ 2$.

Planteamiento. $p = \mathrm{softmax}(z/\tau)$; $H = -\sum p_i \ln p_i$; $\mathrm{Var}_p(z) = \sum p_i z_i^2 - (\sum p_i z_i)^2$.

Desarrollo.

1. $\tau = 0{,}5$: $z/\tau = (6, 2, 0)$; $e^6 = 403{,}4$, $e^2 = 7{,}389$, 1; suma 411,8; $p = (0{,}9796;\ 0{,}0179;\ 0{,}0024)$; $H = 0{,}0202 + 0{,}0721 + 0{,}0146 = 0{,}1069$.
2. $\tau = 1$: $e^3 = 20{,}09$, $e^1 = 2{,}718$, 1; suma 23,80; $p = (0{,}8438;\ 0{,}1142;\ 0{,}0420)$; $H = 0{,}1433 + 0{,}2478 + 0{,}1332 = 0{,}5243$.
3. $\tau = 2$: $z/\tau = (1{,}5;\ 0{,}5;\ 0)$; $e^{1{,}5} = 4{,}482$, $e^{0{,}5} = 1{,}649$, 1; suma 7,130; $p = (0{,}6285;\ 0{,}2312;\ 0{,}1402)$; $H = 0{,}2919 + 0{,}3386 + 0{,}2755 = 0{,}9060$.
4. Varianza con $\tau = 1$: $\mu = 3{\cdot}0{,}8438 + 1{\cdot}0{,}1142 = 2{,}646$; $\sum p_i z_i^2 = 9{\cdot}0{,}8438 + 0{,}1142 = 7{,}708$; $\mathrm{Var} = 7{,}708 - 2{,}646^2 = 7{,}708 - 6{,}999 = 0{,}7093$.
5. Derivada en $b = 1$: $dH/db = -1 \cdot 0{,}7093 = -0{,}7093$.

Resultado. $H = 0{,}107$**,** $0{,}524$ **y** $0{,}906$ **nats para** $\tau = 0{,}5$**,** 1 **y** 2**;** $\mathrm{Var}_p(z) = 0{,}709$ **y** $dH/db = -0{,}709$ **en** $b = 1$**.**

Comprobación. la entropía crece con τ y queda por debajo de $\ln 3 = 1{,}099$. La derivada predice, al pasar de $b = 1$ a $b = 0{,}5$ ($\tau = 2$), un aumento de unos $0{,}709 \cdot 0{,}5 = 0{,}35$ nats en primera aproximación; el aumento real es 0,38, mayor porque la varianza crece al disminuir b.

Solución del ejercicio 5.4

Datos. $q = (0{,}5;\ 0{,}2;\ 0{,}15;\ 0{,}1;\ 0{,}05)$, ya ordenada.

Planteamiento. proposición 5.8: acumular hasta alcanzar ρ, conservar esos tokens y renormalizar. La temperatura $\tau = 0{,}5$ sobre logits $\ln q_i$ eleva cada q_i al cuadrado antes de normalizar.

Desarrollo.

1. Acumuladas: 0,5; 0,7; 0,85; 0,95; 1.
2. $\rho = 0{,}8$: la primera acumulada $\geq 0{,}8$ es 0,85, con 3 tokens. Dividiendo por 0,85: (0,5882; 0,2353; 0,1765; 0; 0).
3. $\rho = 0{,}9$: la primera $\geq 0{,}9$ es 0,95, con 4 tokens. Dividiendo por 0,95: (0,5263; 0,2105; 0,1579; 0,1053; 0).
4. (b) $q_i^2 = (0{,}25;\ 0{,}04;\ 0{,}0225;\ 0{,}01;\ 0{,}0025)$, de suma 0,325; normalizada: (0,7692; 0,1231; 0,0692; 0,0308; 0,0077).
5. Acumuladas: 0,7692; $0{,}8923 \geq 0{,}8$: 2 tokens. Dividiendo por 0,8923: (0,8621; 0,1379; 0; 0; 0).

Resultado. $\rho = 0{,}8$**:** (0,588; 0,235; 0,176; 0; 0)**;** $\rho = 0{,}9$**:** (0,526; 0,211; 0,158; 0,105; 0)**; con** $\tau = 0{,}5$ **y** $\rho = 0{,}8$ **quedan dos tokens:** (0,862; 0,138; 0; 0; 0)**.**

Comprobación. cada resultado suma 1. Al bajar la temperatura la masa se concentra y el núcleo pasa de 3 a 2 tokens: el número de candidatos se adapta a la forma de la distribución.

Solución del ejercicio 5.5

Datos. $\tau > 0$; logits z; para (b), la distribución q del ejercicio 5.4 con $\rho = 0{,}8$ y $\tau = 0{,}5$.

Planteamiento. la división por $\tau > 0$ conserva el orden de los logits; el núcleo, en cambio, depende de la forma de la distribución.

Desarrollo.

1. (a) Como $\tau > 0$, $z_i > z_j$ si y solo si $z_i/\tau > z_j/\tau$, y los empates se conservan; luego $I_k(z/\tau) = I_k(z)$.
2. Top-k y después temperatura: $\mathrm{softmax}\big(\mathrm{top}_k(z)/\tau\big)$, con $-\infty/\tau = -\infty$. Temperatura y después top-k: $\mathrm{softmax}\big(\mathrm{top}_k(z/\tau)\big)$. En ambos casos el resultado es $e^{z_i/\tau} / \sum_{j \in I_k(z)} e^{z_j/\tau}$ en $I_k(z)$ y 0 fuera: coinciden.
3. (b) Temperatura y después top-p: por el ejercicio 5.4, (0,8621; 0,1379; 0; 0; 0).
4. Top-p con $\tau = 1$ y después temperatura: el núcleo es $\{1, 2, 3\}$; dentro de él, $q_i^2 = (0{,}25;\ 0{,}04;\ 0{,}0225)$, de suma 0,3125, y la distribución es (0,8; 0,128; 0,072; 0; 0).
5. Las dos distribuciones son distintas: incluso el número de tokens con probabilidad positiva difiere (2 frente a 3).

6. (c) En la definición 5.7, q es la softmax con temperatura: primero se aplica la temperatura y después se calcula el núcleo, como en CS336 [11, §6].

Resultado. **(a) top-k y temperatura conmutan porque $\tau > 0$ no altera el orden de los logits; (b) con top-p no conmutan:** (0,862; 0,138; 0; 0; 0) **frente a** (0,8; 0,128; 0,072; 0; 0)**; (c) temperatura primero.**

Comprobación. $0{,}8 + 0{,}128 + 0{,}072 = 1$ y $0{,}8621 + 0{,}1379 = 1$.

Solución del ejercicio 5.6

Datos. $T = 8$ da 0,694 nats y $T = 32$ da 0,324; las líneas miden entre unos 58 y 80 caracteres y empiezan por su etiqueta.

Planteamiento. identificar qué información útil queda a más de 8 caracteres de la posición que se predice.

Desarrollo.

1. La etiqueta («receta.», «objeto.», «diálogo.», «parte del tiempo.») está al principio de la línea; a partir del noveno carácter queda fuera de la ventana.
2. Sin la etiqueta, el modelo no sabe qué plantilla escribe: ante «sirve para» solo ve «e␣para␣» y no sabe si sigue el uso de un objeto o el final de una receta.
3. La concordancia de género depende del sustantivo («un reloj negro»), que suele estar a más de 8 caracteres del adjetivo.
4. Las partes fijas de las plantillas («grados y máxima de») se siguen prediciendo bien: 8 caracteres bastan para reconocerlas.
5. Exceso de pérdida: $0{,}694 - 0{,}324 = 0{,}370$ nats por carácter; en una línea de 58 caracteres, $0{,}370 \cdot 58 = 21{,}5$ nats.
6. La etiqueta sola aporta $\ln 4 = 1{,}386$ nats; el exceso es unas 15 veces mayor, porque la incertidumbre sobre la plantilla reaparece en cada elección que depende de ella.

Resultado. **con $T = 8$ el modelo pierde la etiqueta y el sustantivo del que dependen las elecciones: las palabras fijas de las plantillas siguen siendo predecibles, pero colores, materiales, usos y concordancias se vuelven inciertos; el exceso, unos** 21,5 **nats por línea, supera con mucho los** 1,386 **nats de la etiqueta.**

Comprobación. la pérdida con $T = 8$ sigue muy por debajo de la del bigrama (2,035): un contexto corto ya permite completar palabras.

Bibliografía

[1] T. M. Cover y J. A. Thomas. *Elements of Information Theory*, 2.ª ed. Wiley, 2006. Cap. 2, «Entropy, Relative Entropy, and Mutual Information».

[2] A. Fan, M. Lewis e Y. Dauphin. Hierarchical neural story generation. En *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2018. https://arxiv.org/abs/1805.04833

[3] Gemini Team, Google. Gemini: A family of highly capable multimodal models. Informe técnico, 2023. https://arxiv.org/abs/2312.11805

[4] I. Goodfellow, Y. Bengio y A. Courville. *Deep Learning*. MIT Press, 2016. Cap. 7, «Regularization for Deep Learning»; cap. 8, «Optimization for Training Deep Models»; cap. 11, «Practical Methodology». https://www.deeplearningbook.org/

[5] A. Holtzman, J. Buys, L. Du, M. Forbes e Y. Choi. The curious case of neural text degeneration. En *International Conference on Learning Representations (ICLR)*, 2020. https://arxiv.org/abs/1904.09751

[6] A. Karpathy. nanoGPT. Repositorio de software, GitHub, 2022–2025. Archivos `model.py`, `train.py`, `sample.py` y `config/train_shakespeare_char.py`. Desde noviembre de 2025 su autor lo declara obsoleto y remite a nanochat. https://github.com/karpathy/nanoGPT

[7] Llama Team, AI @ Meta. The Llama 3 herd of models. Informe técnico, 2024. https://arxiv.org/abs/2407.21783

[8] I. Loshchilov y F. Hutter. SGDR: Stochastic gradient descent with warm restarts. En *International Conference on Learning Representations (ICLR)*, 2017. https://arxiv.org/abs/1608.03983

[9] I. Loshchilov y F. Hutter. Decoupled weight decay regularization. En *International Conference on Learning Representations (ICLR)*, 2019. https://arxiv.org/abs/1711.05101

[10] A. Radford, J. Wu, R. Child, D. Luan, D. Amodei e I. Sutskever. Language models are unsupervised multitask learners. Informe técnico, OpenAI, 2019.

[11] Stanford CS336. Language Modeling from Scratch. Stanford University, primavera de 2026. Tarea 1 (*basics*), «Building a Transformer LM», versión 26.0.3. https://github.com/stanford-cs336/assignment1-basics

[12] H. Touvron *et al.* LLaMA: Open and efficient foundation language models. Prepublicación, 2023. https://arxiv.org/abs/2302.13971

[13] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser e I. Polosukhin. Attention is all you need. En *Advances in Neural Information Processing Systems 30 (NIPS)*, 2017. https://arxiv.org/abs/1706.03762

[14] xAI. Grok-1: pesos abiertos y código de ejemplo. Repositorio de software, GitHub, 2024. https://github.com/xai-org/grok-1

[15] D. Yang y Y. Choi. CS224N: Natural Language Processing with Deep Learning. Stanford University, invierno de 2026. Clase 5, «Transformers»; clase 7, «Pretraining (Scaling, Systems, Data)». https://web.stanford.edu/class/cs224n/

[16] A. Zhang, Z. C. Lipton, M. Li y A. J. Smola. *Dive into Deep Learning*, versión 1.0.3. Cambridge University Press, 2023. Cap. 9, «Recurrent Neural Networks» (§9.3 y §9.5); cap. 11, «Attention Mechanisms and Transformers» (§11.9). https://d2l.ai/